

The Malice of ELF's

Practical Anamorphic-Resistant Encryption without Random Oracles

Gennaro Avitabile, Vincenzo Botta, Emanuele Giunta,
Marcin (Miriam) Mielniczuk, Francesco Migliaro

- 1 Introduction and definitions
- 2 Construction 1: conceptual
- 3 Proof of Anamorphic Resistance
- 4 Construction 2: practical
- 5 Construction 3: Definitive ARE
- 6 Summary

Anamorphic Encryption [PPY22]



Dictator



Alice

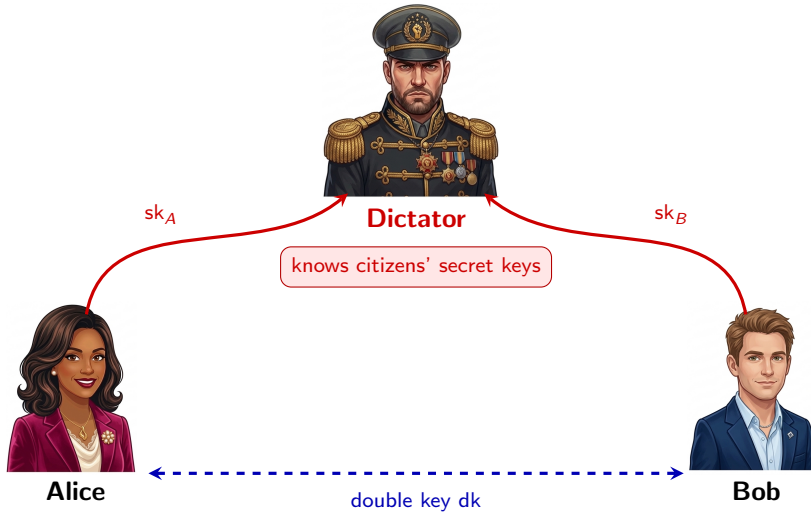


Bob

Anamorphic Encryption [PPY22]



Anamorphic Encryption [PPY22]



Anamorphic Encryption [PPY22]



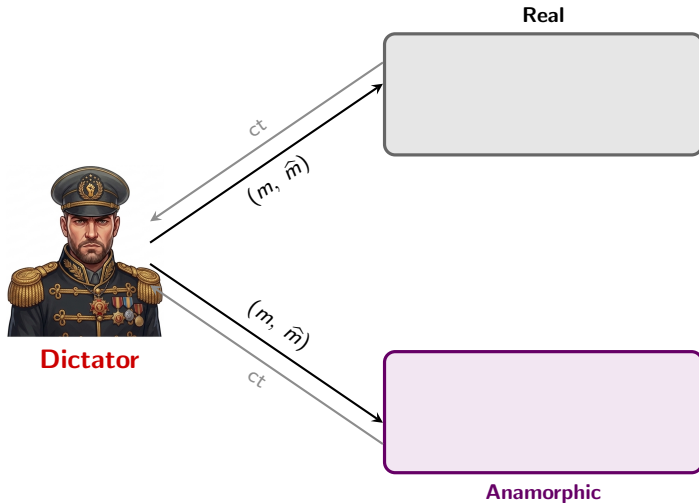
Anamorphic Encryption [PPY22]



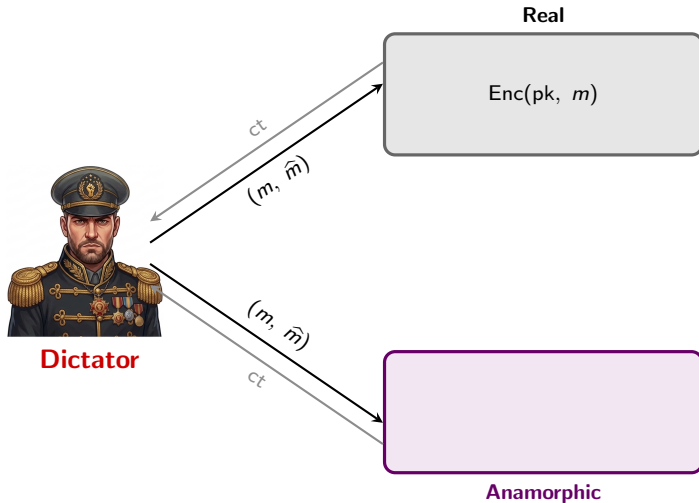
Anamorphic Encryption [PPY22]



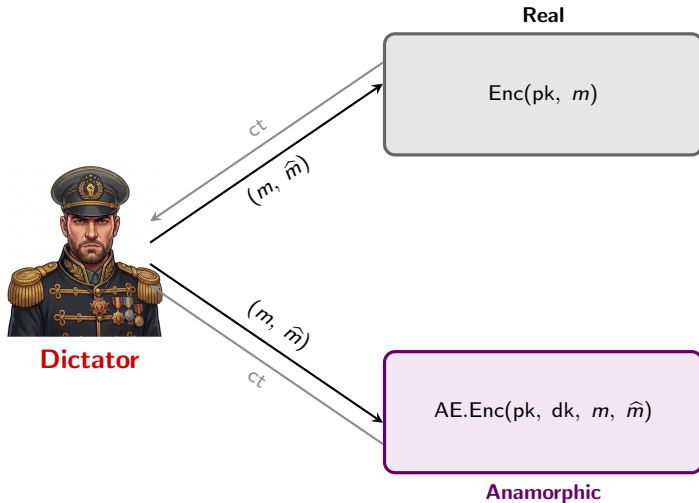
Formal security notion [PPY22]



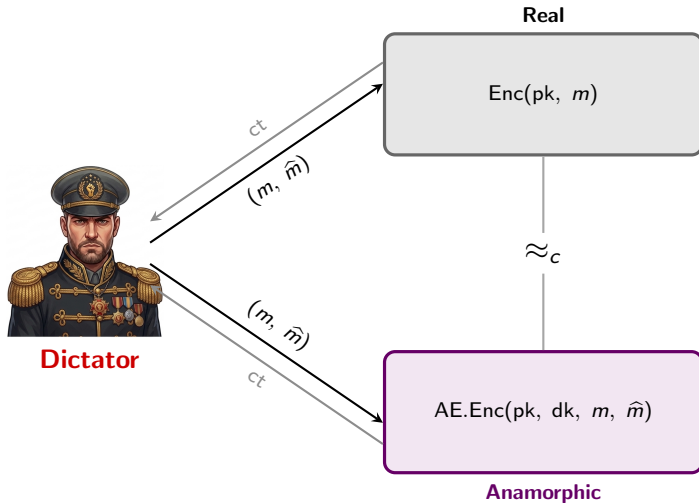
Formal security notion [PPY22]



Formal security notion [PPY22]



Formal security notion [PPY22]



Dictator cannot distinguish Real from Anamorphic

Models of the dictator



Citizens use a standard, well-known Enc.



Alice



Bob

Models of the dictator



Alice



Bob

Models of the dictator



Models of the dictator



Models of the dictator



Models of the dictator



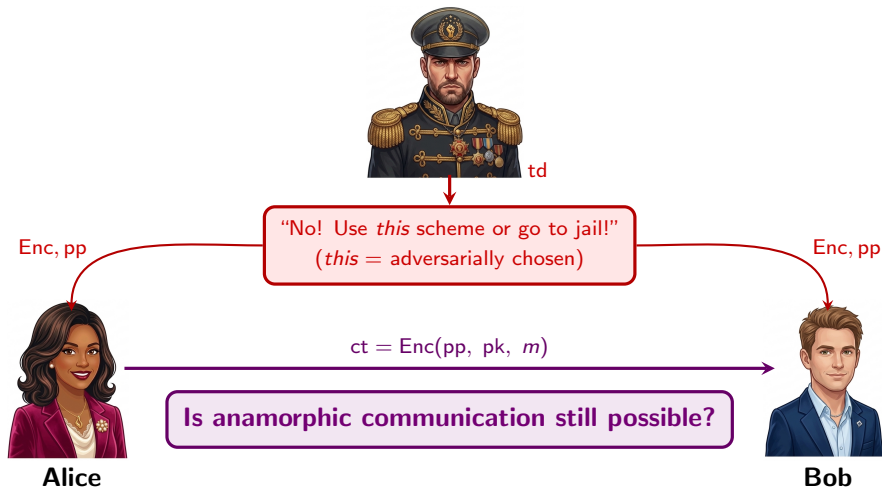
Models of the dictator



Models of the dictator



Models of the dictator



Rejection Sampling: a canonical approach



Alice

sk_A

$$r \leftarrow \$ \{0, 1\}^\lambda$$

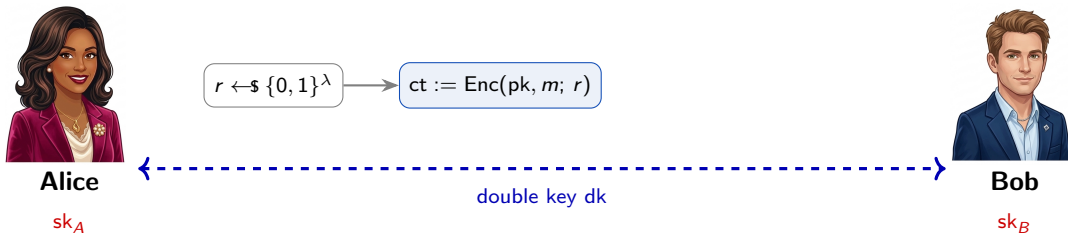


Bob

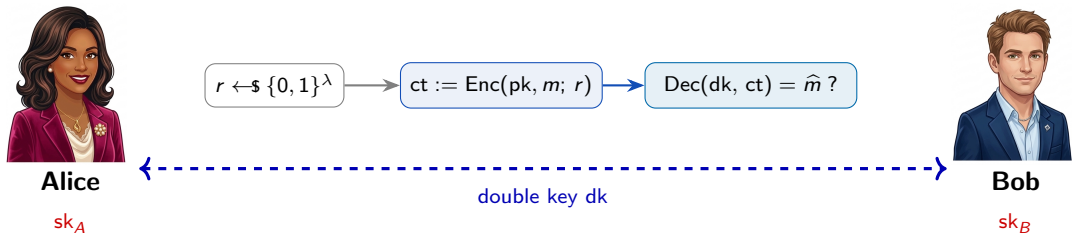
sk_B

double key dk

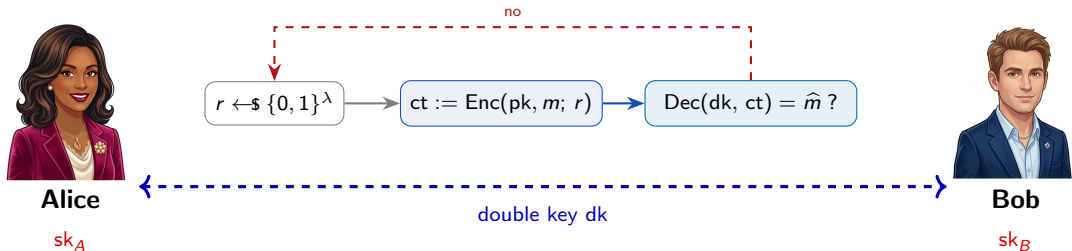
Rejection Sampling: a canonical approach



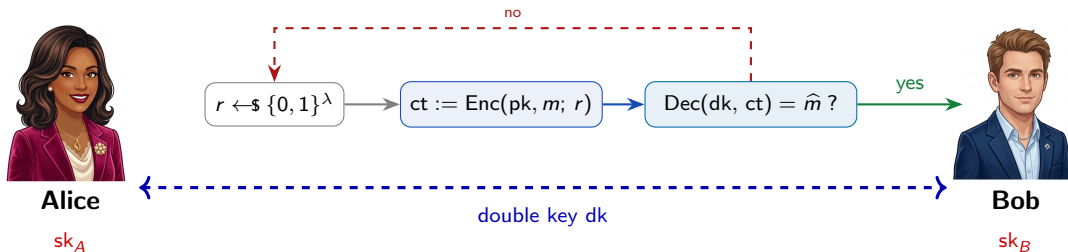
Rejection Sampling: a canonical approach



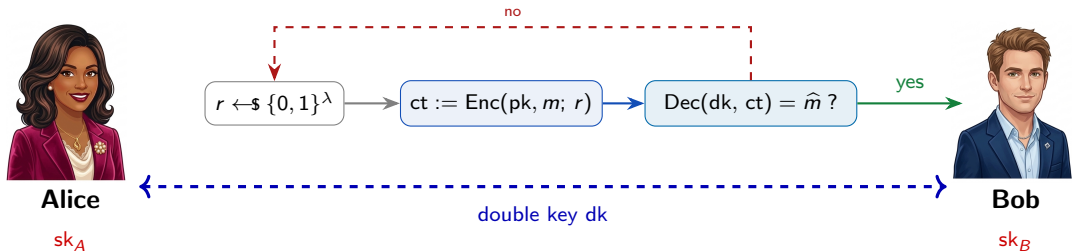
Rejection Sampling: a canonical approach



Rejection Sampling: a canonical approach

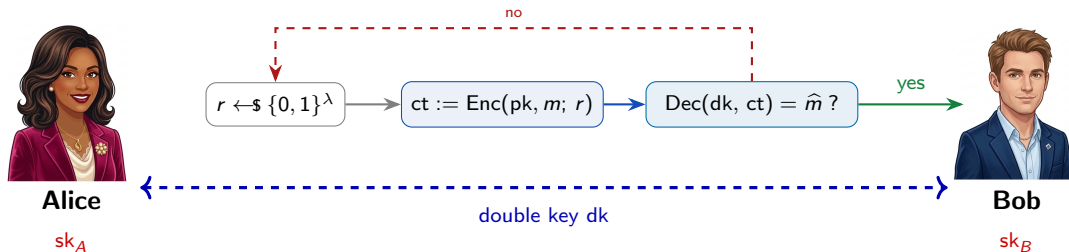


Rejection Sampling: a canonical approach



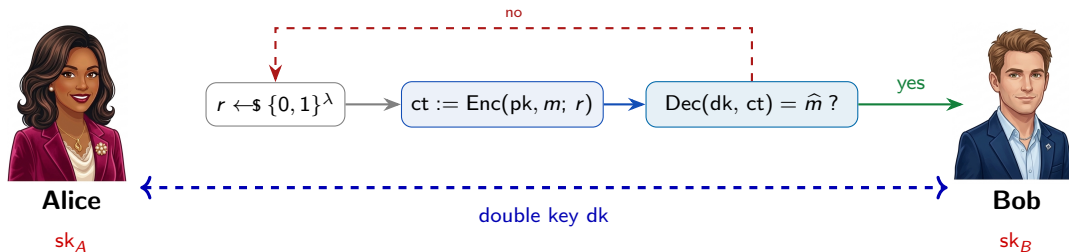
- Staple in steganography.

Rejection Sampling: a canonical approach



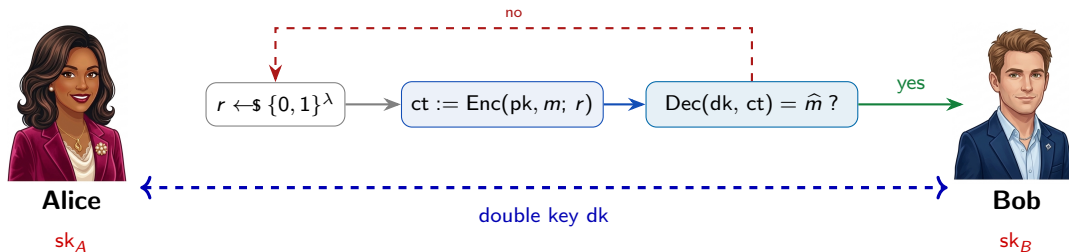
- Staple in steganography.
- **Bad:** can only transmit $O(\log \lambda)$ bits.

Rejection Sampling: a canonical approach



- Staple in steganography.
- **Bad:** can only transmit $O(\log \lambda)$ bits.
- **Good:** a generic way to obtain AE.

Rejection Sampling: a canonical approach

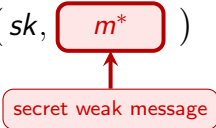


- Staple in steganography.
- **Bad:** can only transmit $O(\log \lambda)$ bits.
- **Good:** a generic way to obtain AE.
- Is it?

Breaking Rejection Sampling [CGM24]

$$sk^* = (sk, \boxed{})$$

Breaking Rejection Sampling [CGM24]

$$sk^* = (sk, m^*)$$


secret weak message

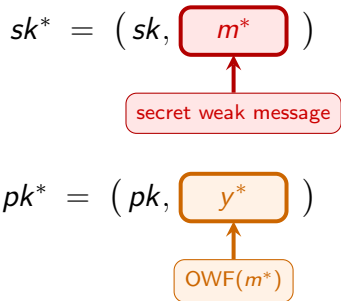
Breaking Rejection Sampling [CGM24]

$$sk^* = (sk, m^*)$$

secret weak message

$$pk^* = (pk, \quad)$$

Breaking Rejection Sampling [CGM24]



Breaking Rejection Sampling [CGM24]

$$sk^* = (sk, m^*)$$

secret weak message

$$pk^* = (pk, y^*)$$

OWF(m^*)

encrypt message m

OWF(m) = y^* ?

Breaking Rejection Sampling [CGM24]

$$sk^* = (sk, m^*)$$

secret weak message

$$pk^* = (pk, y^*)$$

OWF(m^*)

encrypt message m

OWF(m) = y^* ?

yes

$c \leftarrow \$ B$ ($|B| = \text{poly}(\lambda)$)

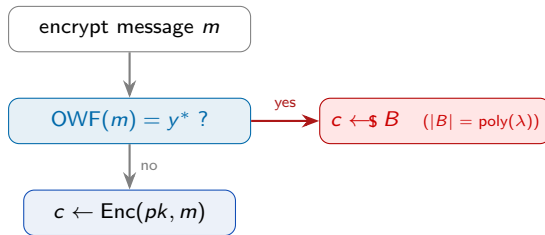
Breaking Rejection Sampling [CGM24]

$$sk^* = (sk, m^*)$$

secret weak message

$$pk^* = (pk, y^*)$$

$OWF(m^*)$



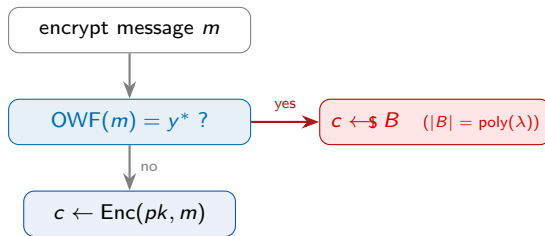
Breaking Rejection Sampling [CGM24]

$$sk^* = (sk, m^*)$$

secret weak message

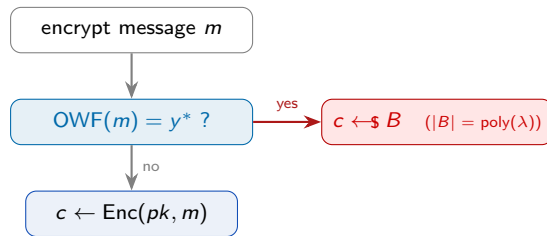
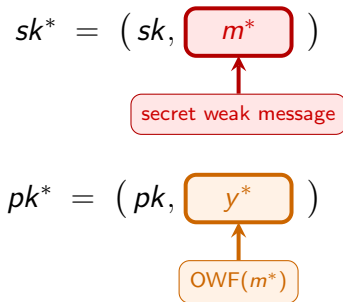
$$pk^* = (pk, y^*)$$

OWF(m^*)



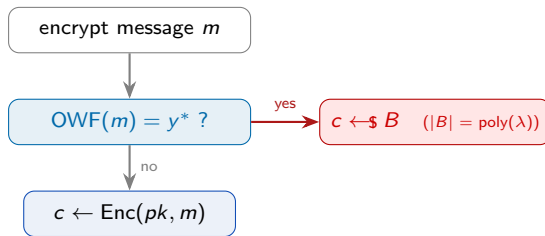
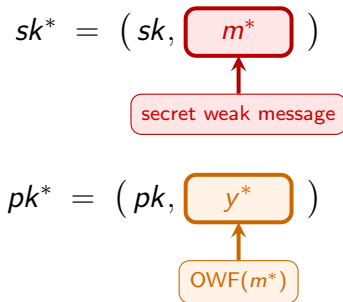
- IND-CPA holds: finding m^* from pk^* requires inverting OWF.

Breaking Rejection Sampling [CGM24]



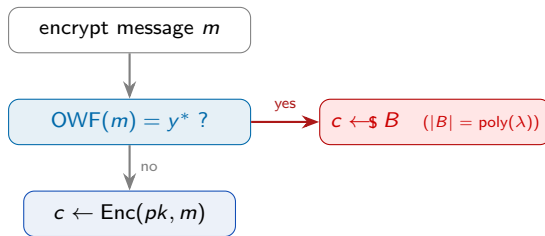
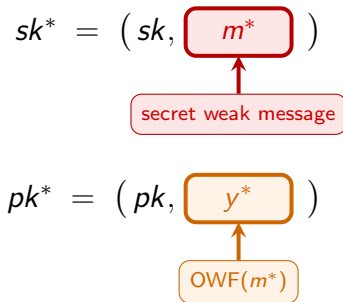
- IND-CPA holds: finding m^* from pk^* requires inverting OWF.
- AE doesn't:

Breaking Rejection Sampling [CGM24]



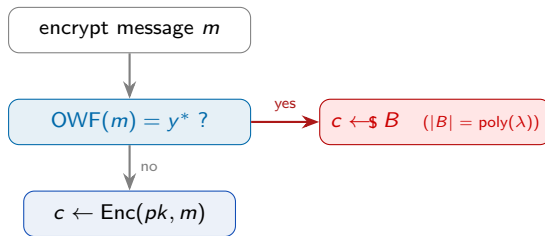
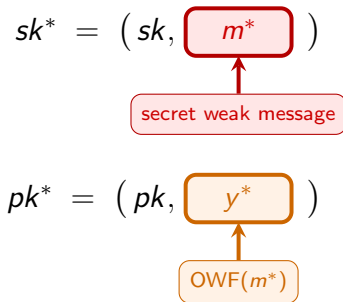
- IND-CPA holds: finding m^* from pk^* requires inverting OWF.
- AE doesn't:
 - Dictator can read m^* from sk^* .

Breaking Rejection Sampling [CGM24]



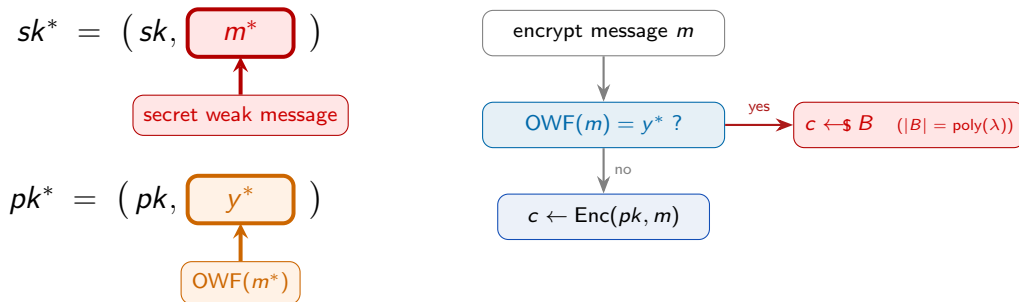
- IND-CPA holds: finding m^* from pk^* requires inverting OWF.
- AE doesn't:
 - Dictator can read m^* from sk^* .
 - Let $c_i = \text{AE.Enc}(pk, m^*, \hat{m} = b)$.

Breaking Rejection Sampling [CGM24]



- IND-CPA holds: finding m^* from pk^* requires inverting OWF.
- AE doesn't:
 - Dictator can read m^* from sk^* .
 - Let $c_i = \text{AE.Enc}(pk, m^*, \hat{m} = b)$.
 - Normal mode: $c_0 = c_1$ with noticeable probability

Breaking Rejection Sampling [CGM24]



- IND-CPA holds: finding m^* from pk^* requires inverting OWF.
- AE doesn't:
 - Dictator can read m^* from sk^* .
 - Let $c_i = \text{AE.Enc}(pk, m^*, \hat{m} = b)$.
 - Normal mode: $c_0 = c_1$ with noticeable probability
 - Anamorphic mode: $c_0 \neq c_1$ by correctness.

Not quite: for some (contrived) schemes, rejection sampling does not achieve AE.

Not quite: for some (contrived) schemes, rejection sampling does not achieve AE.

More generally, black-box AE is impossible.

Not quite: for some (contrived) schemes, rejection sampling does not achieve AE.

More generally, black-box AE is impossible.

Semi-adaptive security:

One can weaken the security definition so that rejection sampling works.

Not quite: for some (contrived) schemes, rejection sampling does not achieve AE.

More generally, black-box AE is impossible.

Semi-adaptive security:

One can weaken the security definition so that rejection sampling works.

Still meaningful!

Limits of Black-Box AE [CGM24]

Not quite: for some (contrived) schemes, rejection sampling does not achieve AE.

More generally, black-box AE is impossible.

Semi-adaptive security:

One can weaken the security definition so that rejection sampling works.

Still meaningful!

Impossibility: even under semi-adaptive security, any black-box AE can transmit $O(\log \lambda)$ bits per ciphertext

Limits of Black-Box AE [CGM24]

Not quite: for some (contrived) schemes, rejection sampling does not achieve AE.

More generally, black-box AE is impossible.

Semi-adaptive security:

One can weaken the security definition so that rejection sampling works.

Still meaningful!

Impossibility: even under semi-adaptive security, any black-box AE can transmit $O(\log \lambda)$ bits per ciphertext

Rejection sampling is optimal.

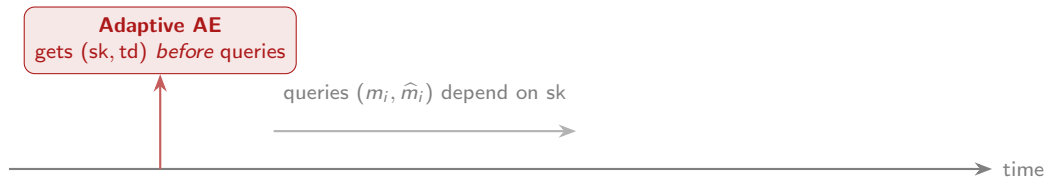
Two flavors of AE [CGM24]

→ time

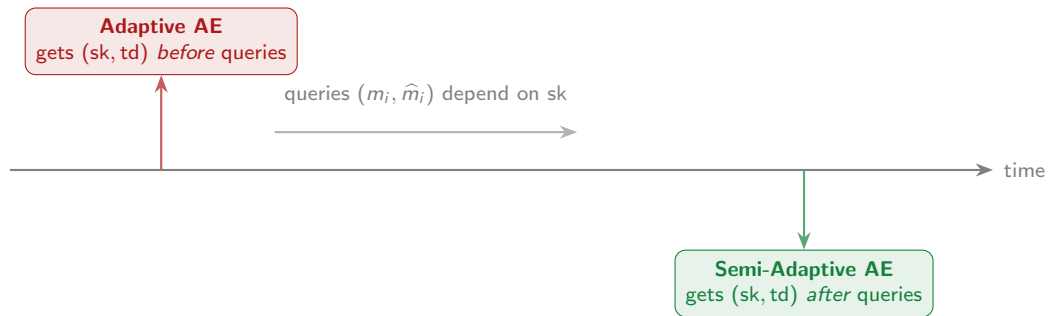
Two flavors of AE [CGM24]



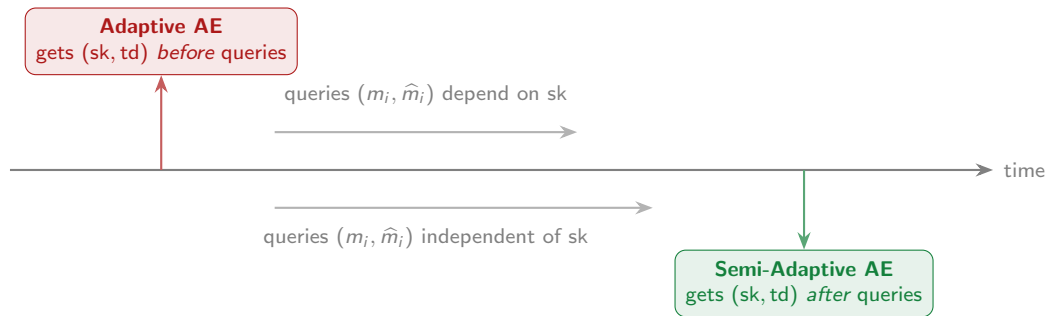
Two flavors of AE [CGM24]



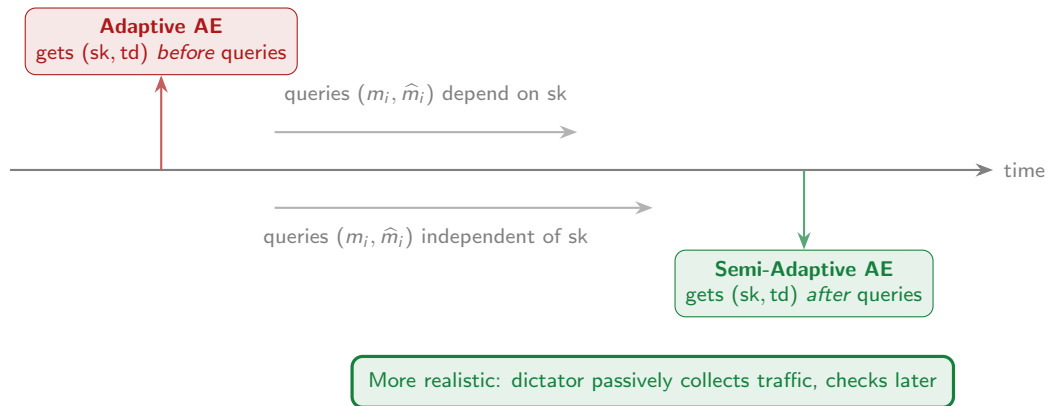
Two flavors of AE [CGM24]



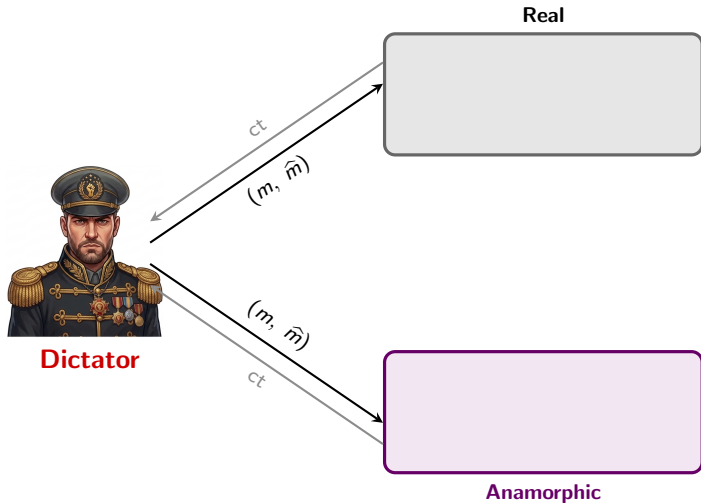
Two flavors of AE [CGM24]



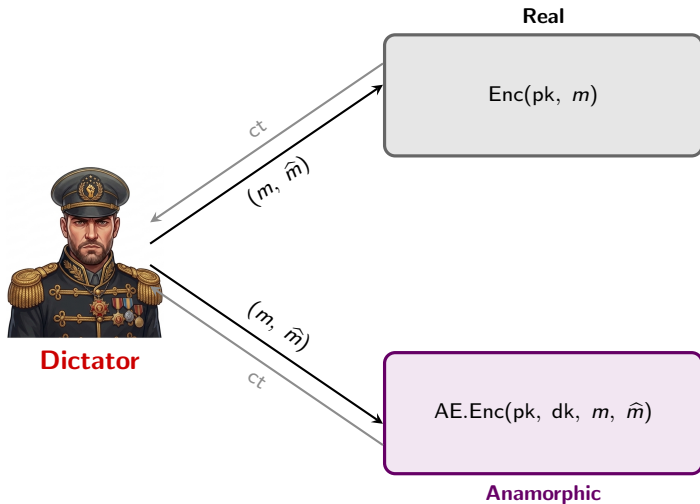
Two flavors of AE [CGM24]



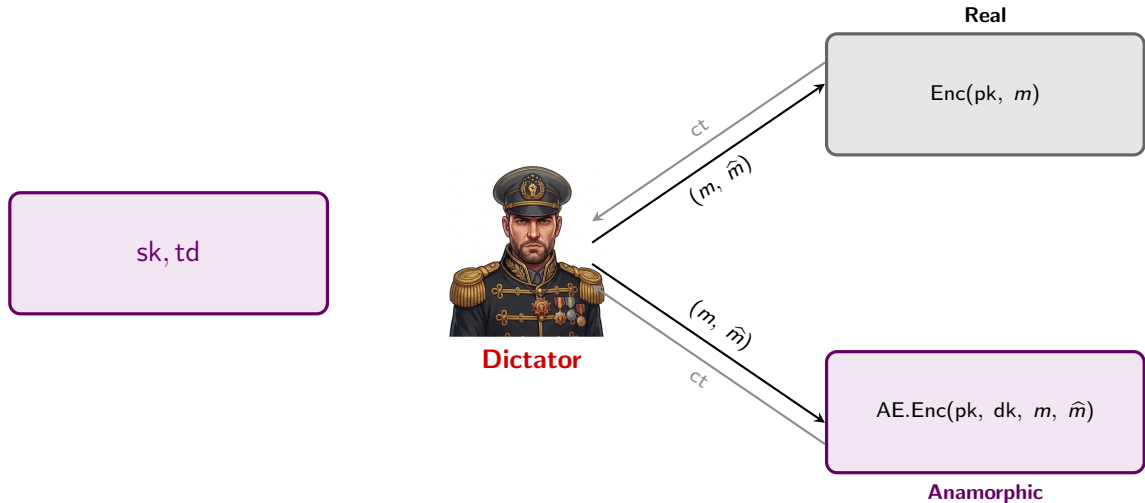
Semi-adaptive AE: definition [CGM24]



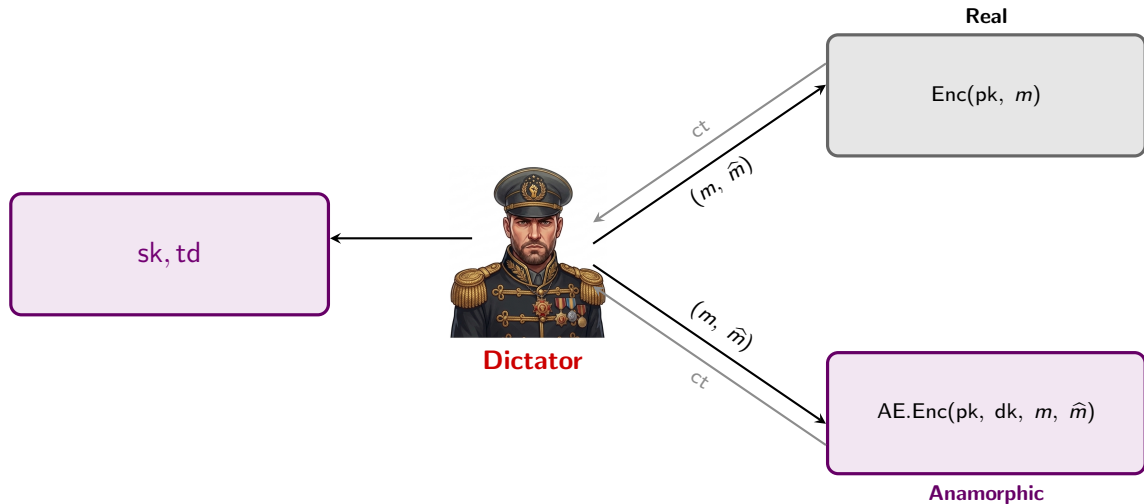
Semi-adaptive AE: definition [CGM24]



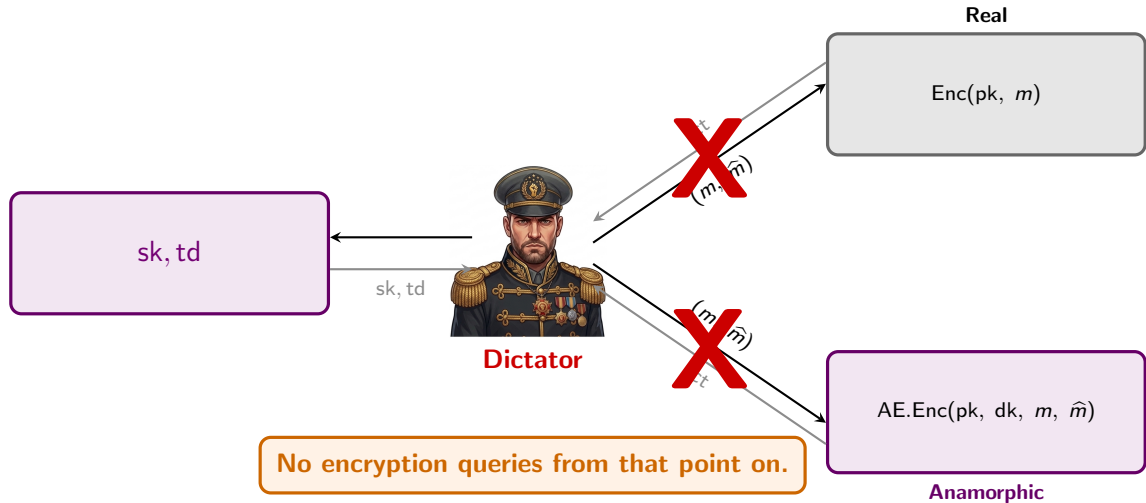
Semi-adaptive AE: definition [CGM24]



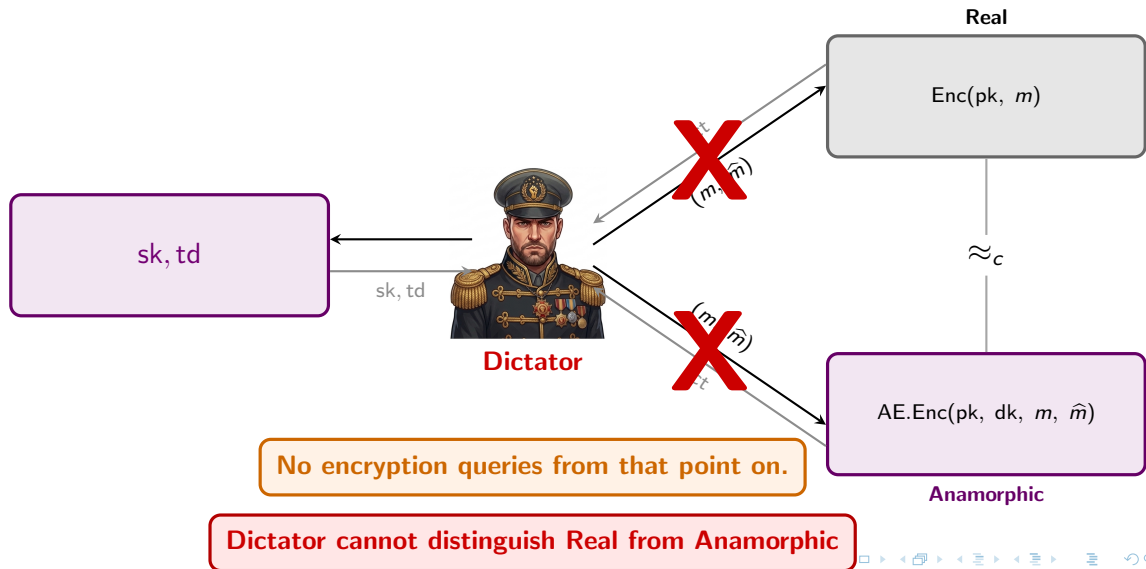
Semi-adaptive AE: definition [CGM24]



Semi-adaptive AE: definition [CGM24]



Semi-adaptive AE: definition [CGM24]



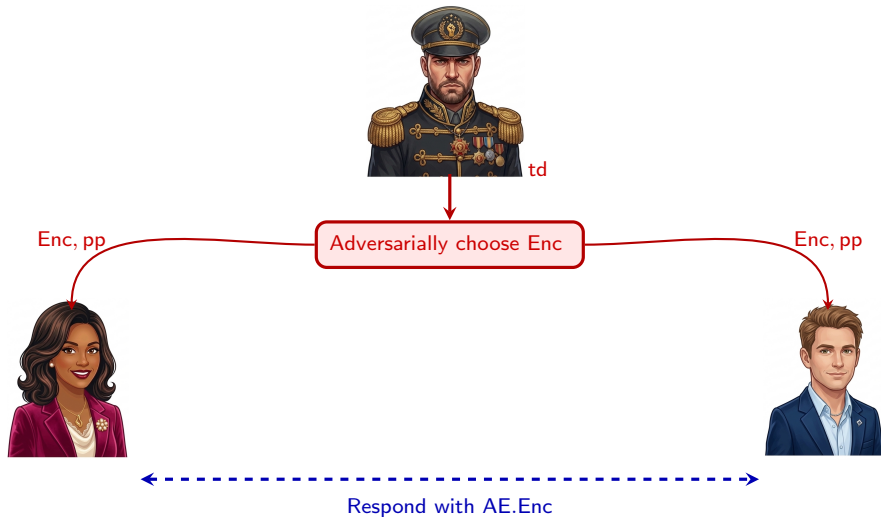
What is anamorphic resistance? [DG25]



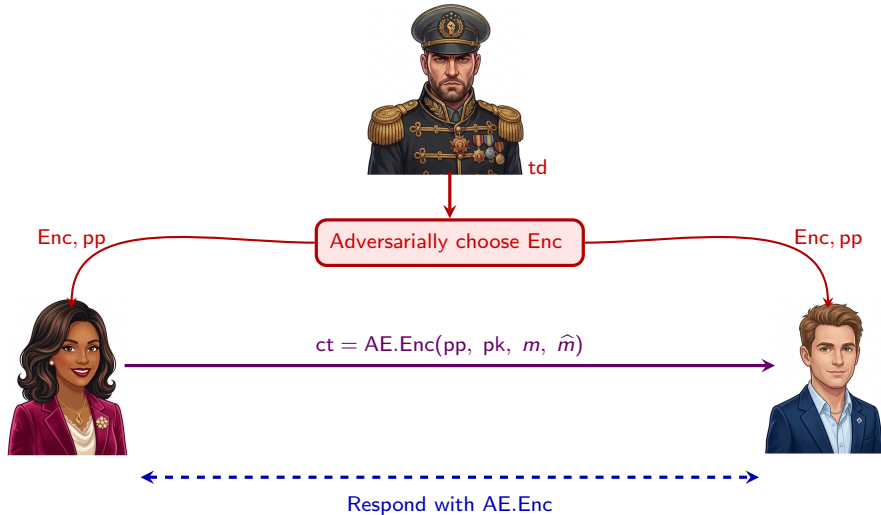
What is anamorphic resistance? [DG25]



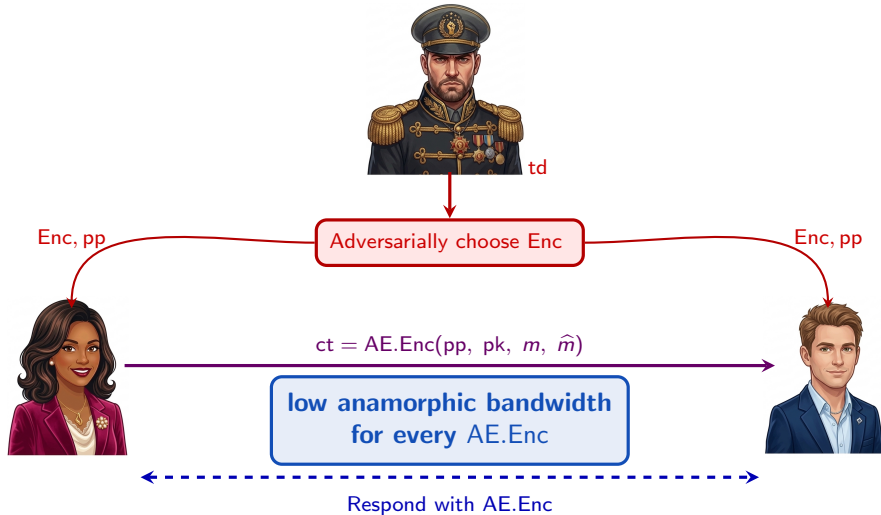
What is anamorphic resistance? [DG25]



What is anamorphic resistance? [DG25]



What is anamorphic resistance? [DG25]



Prior work and our contributions

ARE	AE model	Assumption	AE bandwidth
-----	----------	------------	--------------

Prior work and our contributions

ARE	AE model	Assumption	AE bandwidth
[DG25]	Semi-Adaptive	ROM	$O(\log \lambda)$

Prior work and our contributions

ARE	AE model	Assumption	AE bandwidth
[DG25]	Semi-Adaptive	ROM	$O(\log \lambda)$
[CGGM25] Construction 1	Adaptive	$i\mathcal{O}$	0

Prior work and our contributions

ARE	AE model	Assumption	AE bandwidth
[DG25]	Semi-Adaptive	ROM	$O(\log \lambda)$
[CGGM25] Construction 1	Adaptive	$i\mathcal{O}$	0
[CGGM25] Construction 2	Adaptive	ROM	0

Prior work and our contributions

ARE	AE model	Assumption	AE bandwidth
[DG25]	Semi-Adaptive	ROM	$O(\log \lambda)$
[CGGM25] Construction 1	Adaptive	$i\mathcal{O}$	0
[CGGM25] Construction 2	Adaptive	ROM	0
Construction 1	Semi-Adaptive	$\text{expDDH} + i\mathcal{O}$	$O(\log \lambda)$ conceptual

Prior work and our contributions

ARE	AE model	Assumption	AE bandwidth	
[DG25]	Semi-Adaptive	ROM	$O(\log \lambda)$	
[CGGM25] Construction 1	Adaptive	$i\mathcal{O}$	0	
[CGGM25] Construction 2	Adaptive	ROM	0	
Construction 1	Semi-Adaptive	$\text{expDDH} + i\mathcal{O}$	$O(\log \lambda)$	conceptual
Construction 2	Semi-Adaptive	expDDH	$O(\log \lambda)$	practical

Prior work and our contributions

ARE	AE model	Assumption	AE bandwidth	
[DG25]	Semi-Adaptive	ROM	$O(\log \lambda)$	
[CGGM25] Construction 1	Adaptive	$i\mathcal{O}$	0	
[CGGM25] Construction 2	Adaptive	ROM	0	
Construction 1	Semi-Adaptive	$\text{expDDH} + i\mathcal{O}$	$O(\log \lambda)$	conceptual
Construction 2	Semi-Adaptive	expDDH	$O(\log \lambda)$	practical
Construction 3	Semi-Adaptive	$\text{expDDH} + i\mathcal{O}$	$O(\log \lambda)$	theoretically
	Adaptive		0	optimal

Prior work and our contributions

ARE	AE model	Assumption	AE bandwidth	
[DG25]	Semi-Adaptive	ROM	$O(\log \lambda)$	
[CGGM25] Construction 1	Adaptive	$i\mathcal{O}$	0	
[CGGM25] Construction 2	Adaptive	ROM	0	
Construction 1	Semi-Adaptive	$\text{expDDH} + i\mathcal{O}$	$O(\log \lambda)$	conceptual
Construction 2	Semi-Adaptive	expDDH	$O(\log \lambda)$	practical
Construction 3	Semi-Adaptive	$\text{expDDH} + i\mathcal{O}$	$O(\log \lambda)$	theoretically
	Adaptive		0	optimal

ROM caveat for ARE:
 AE could use the hash *non-black-box*

- 1 Introduction and definitions
- 2 Construction 1: conceptual
- 3 Proof of Anamorphic Resistance
- 4 Construction 2: practical
- 5 Construction 3: Definitive ARE
- 6 Summary

Where to hide the message?

$\text{Enc}(\boxed{\text{pp}}, \boxed{\text{pk}}, \boxed{m}; \boxed{r})$

Where to hide the message?

$\text{Enc}(\boxed{\text{pp}}, \boxed{\text{pk}}, \boxed{m}; \boxed{r})$

chosen by the dictator

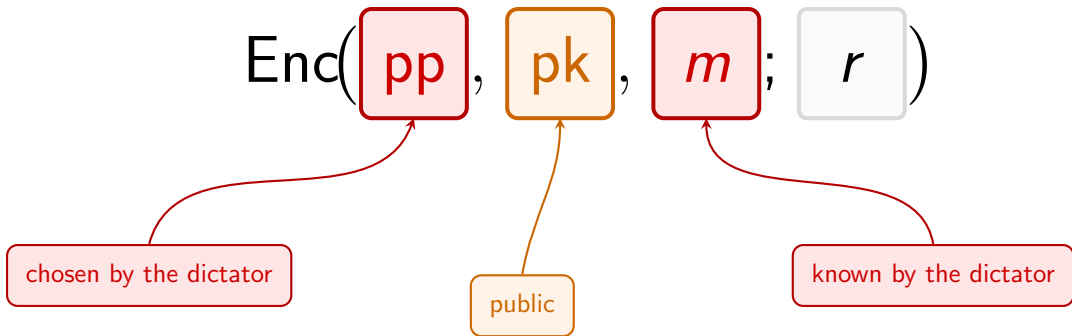
Where to hide the message?

$\text{Enc}(\text{pp}, \text{pk}, m; r)$

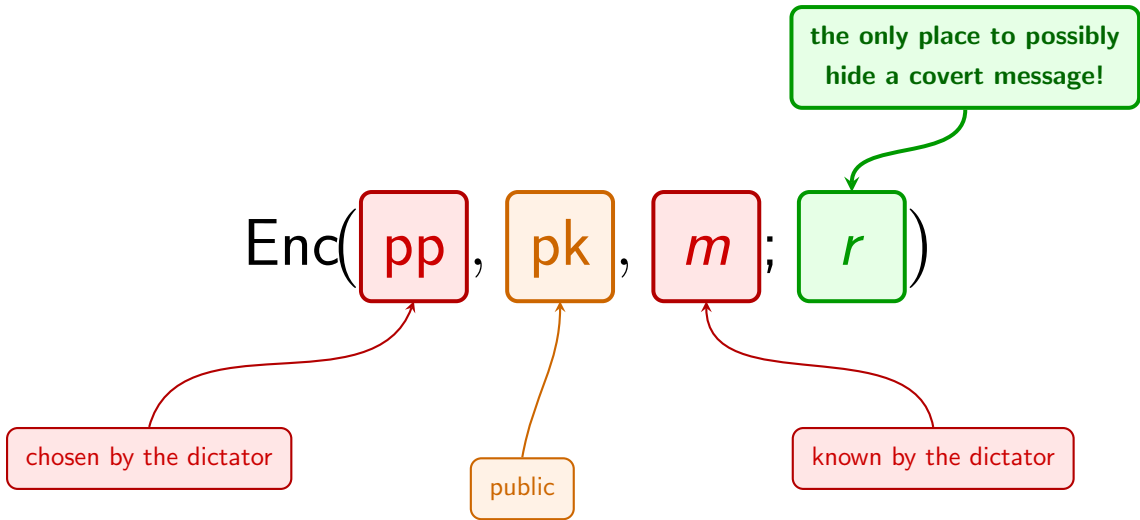
chosen by the dictator

public

Where to hide the message?



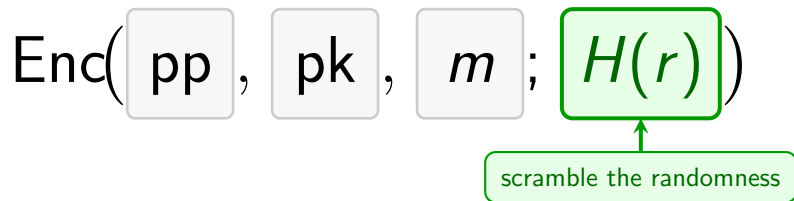
Where to hide the message?



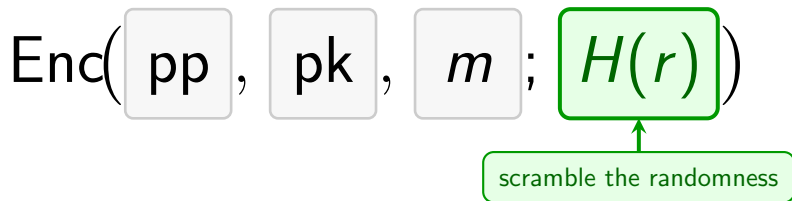
First idea: pass r through a hash

$\text{Enc}(\text{pp}, \text{pk}, m; r)$

First idea: pass r through a hash

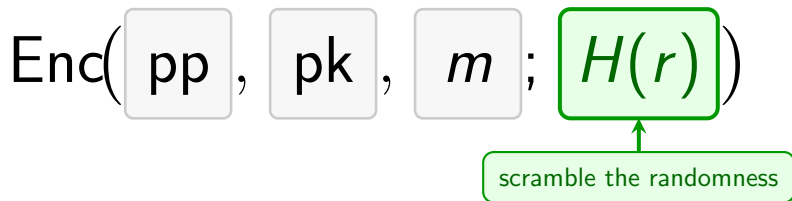


First idea: pass r through a hash



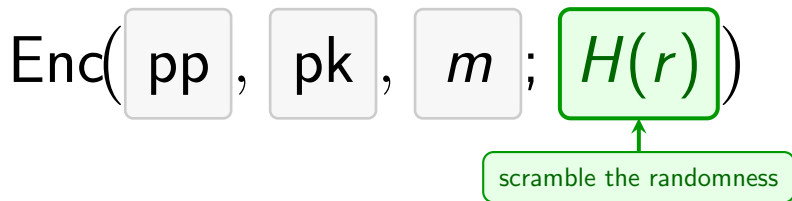
- H is a random oracle $\implies$ can only rejection sample

First idea: pass r through a hash



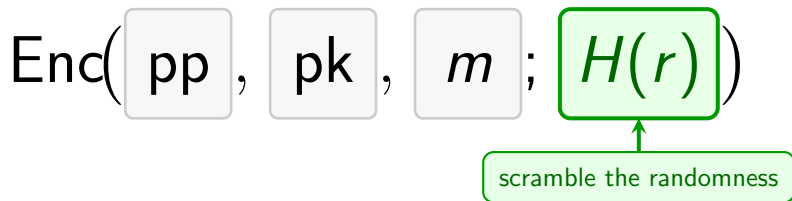
- H is a random oracle $\implies$ can only rejection sample
- Not enforceable!

First idea: pass r through a hash



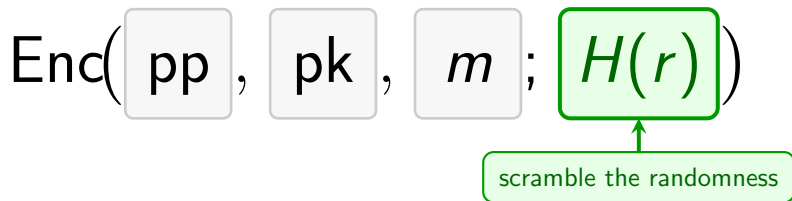
- H is a random oracle $\implies$ can only rejection sample
- **Not enforceable!**
- Natural fix: attach a NIZK proving " $H(r)$ was computed correctly" (or similar)

First idea: pass r through a hash



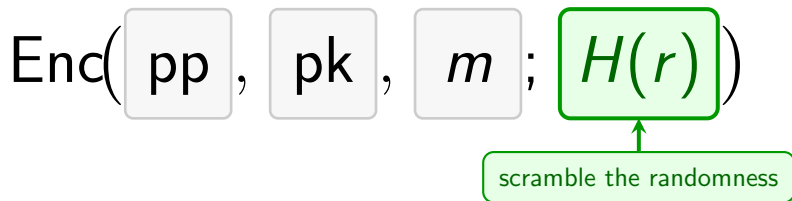
- H is a random oracle $\implies$ can only rejection sample
- **Not enforceable!**
- Natural fix: attach a NIZK proving " $H(r)$ was computed correctly" (or similar)
 - NIZKs are typically randomized

First idea: pass r through a hash



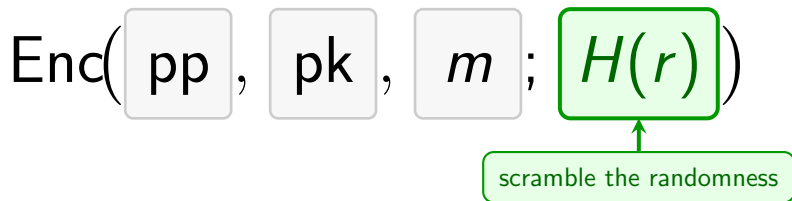
- H is a random oracle $\implies$ can only rejection sample
- **Not enforceable!**
- Natural fix: attach a NIZK proving " $H(r)$ was computed correctly" (or similar)
 - NIZKs are typically randomized $\rightsquigarrow$ back to square one.

First idea: pass r through a hash



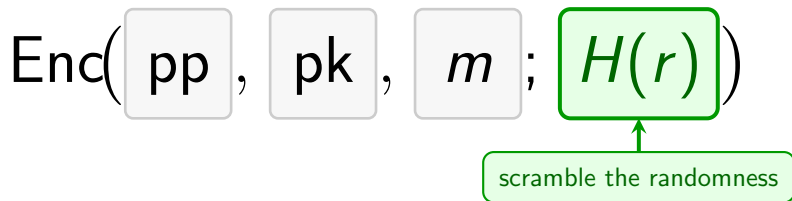
- H is a random oracle $\implies$ can only rejection sample
- **Not enforceable!**
- Natural fix: attach a NIZK proving " $H(r)$ was computed correctly" (or similar)
 - NIZKs are typically randomized $\rightsquigarrow$ back to square one.
 - NIZKs over a Random Oracle???

First idea: pass r through a hash



- H is a random oracle $\implies$ can only rejection sample
- **Not enforceable!**
- Natural fix: attach a NIZK proving " $H(r)$ was computed correctly" (or similar)
 - NIZKs are typically randomized $\rightsquigarrow$ back to square one.
 - NIZKs over a Random Oracle???
- **[DG25]:** ROM solution: use *deterministic encryption* to the dictator's pk.

First idea: pass r through a hash



- H is a random oracle $\implies$ can only rejection sample
- **Not enforceable!**
- Natural fix: attach a NIZK proving " $H(r)$ was computed correctly" (or similar)
 - NIZKs are typically randomized $\rightsquigarrow$ back to square one.
 - NIZKs over a Random Oracle???
- **[DG25]:** ROM solution: use *deterministic encryption* to the dictator's pk.
- **Goal:** achieve this in the **standard model**.

Challenge 1: enforcing adherence

- Suppose H is a circuit emulating a random oracle,

Challenge 1: enforcing adherence

- Suppose H is a circuit emulating a random oracle, compliant ciphertext:

$$\text{ct} = \text{Enc}(\text{pp}, \text{pk}, m; H(r)).$$

Challenge 1: enforcing adherence

- Suppose H is a circuit emulating a random oracle, compliant ciphertext:

$$\text{ct} = \text{Enc}(\text{pp}, \text{pk}, m; H(r)).$$

- If the NIZK for “computed correctly” was **deterministic**, then we’re done.

Challenge 1: enforcing adherence

- Suppose H is a circuit emulating a random oracle, compliant ciphertext:

$$\text{ct} = \text{Enc}(\text{pp}, \text{pk}, m; H(r)).$$

- If the NIZK for “computed correctly” was **deterministic**, then we’re done.
- More accurately, we introduce the following definition:

Challenge 1: enforcing adherence

- Suppose H is a circuit emulating a random oracle, compliant ciphertext:

$$\text{ct} = \text{Enc}(\text{pp}, \text{pk}, m; H(r)).$$

- If the NIZK for “computed correctly” was **deterministic**, then we’re done.
- More accurately, we introduce the following definition:

Fully Unique NIZK

A NIZK is *fully unique* if, for every statement x , there exists **at most one** accepting proof π .

Challenge 1: enforcing adherence

- Suppose H is a circuit emulating a random oracle, compliant ciphertext:

$$\text{ct} = \text{Enc}(\text{pp}, \text{pk}, m; H(r)).$$

- If the NIZK for “computed correctly” was **deterministic**, then we’re done.
- More accurately, we introduce the following definition:

Fully Unique NIZK

A NIZK is *fully unique* if, for every statement x , there exists **at most one** accepting proof π .

- Exist from indistinguishability obfuscation ($i\mathcal{O}$). [WW24,WZ24]

Challenge 1: enforcing adherence

- Suppose H is a circuit emulating a random oracle, compliant ciphertext:

$$\text{ct} = \text{Enc}(\text{pp}, \text{pk}, m; H(r)).$$

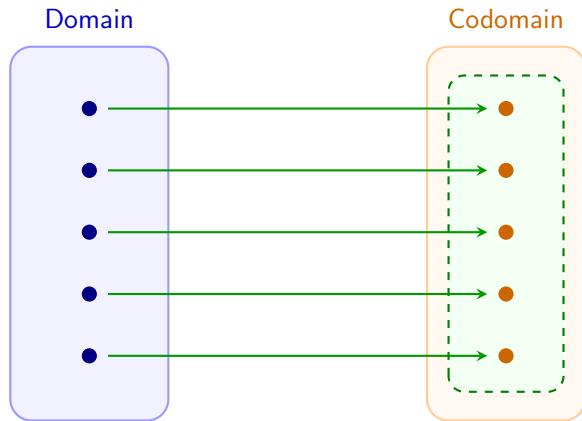
- If the NIZK for “computed correctly” was **deterministic**, then we’re done.
- More accurately, we introduce the following definition:

Fully Unique NIZK

A NIZK is *fully unique* if, for every statement x , there exists **at most one** accepting proof π .

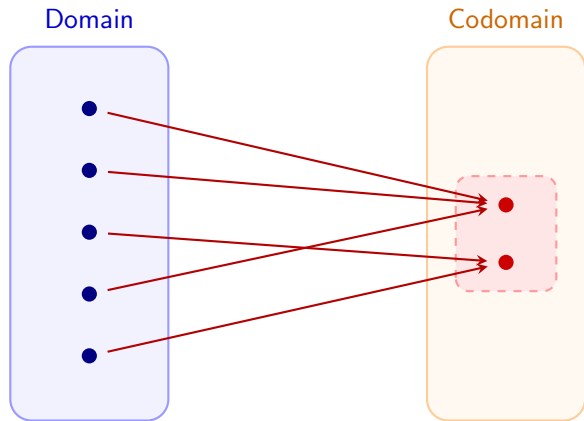
- Exist from indistinguishability obfuscation ($i\mathcal{O}$). [WW24,WZ24]
- Remaining question: **how to emulate the random oracle?**

Key building block: Extremely Lossy Functions (ELFs)



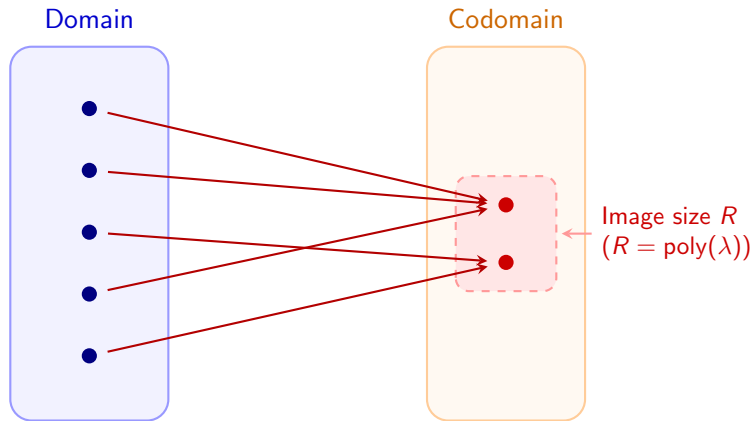
Injective Mode

Key building block: Extremely Lossy Functions (ELFs)



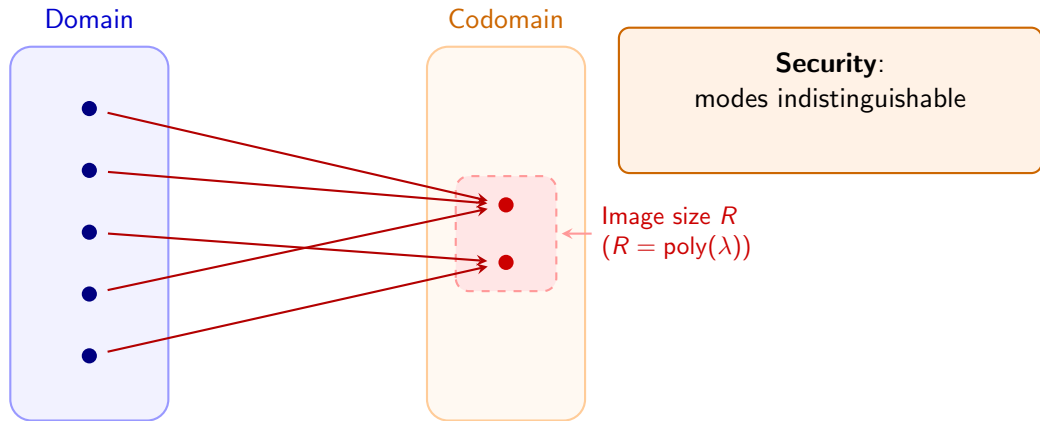
Lossy Mode

Key building block: Extremely Lossy Functions (ELFs)



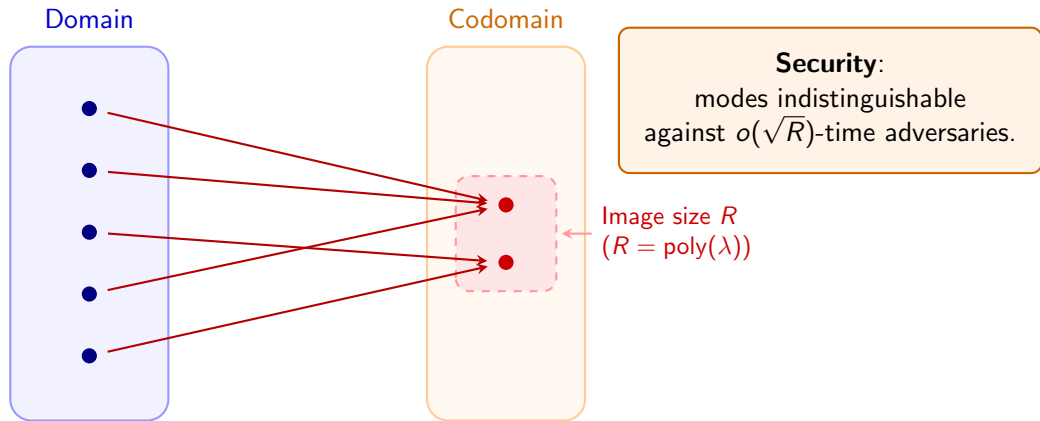
Lossy Mode

Key building block: Extremely Lossy Functions (ELFs)



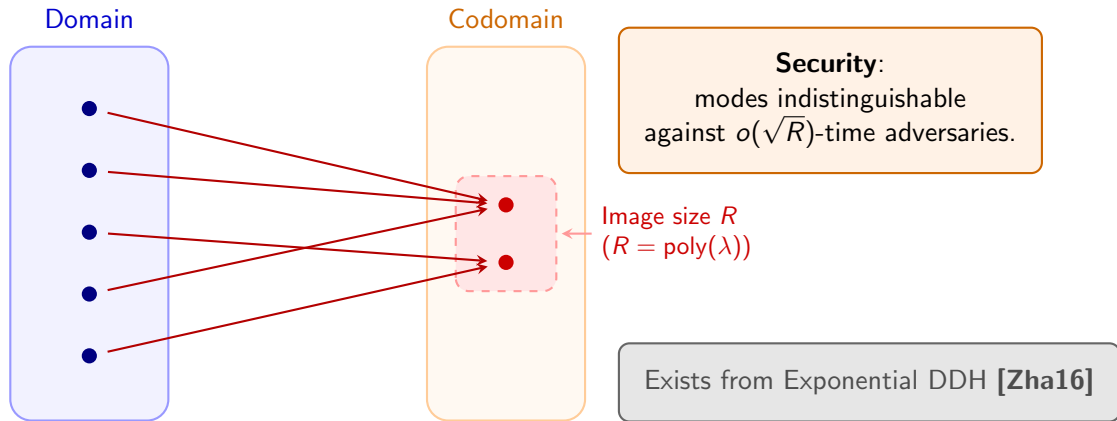
Lossy Mode

Key building block: Extremely Lossy Functions (ELFs)



Lossy Mode

Key building block: Extremely Lossy Functions (ELFs)



Lossy Mode

Challenge 2: instantiating RO

- **Needed property:** “cannot do better than brute-force”.

Challenge 2: instantiating RO

- **Needed property:** “cannot do better than brute-force”.
- Classical formalization:

Correlation Intractability (CI)

For a sufficiently sparse relation $\mathcal{R}$, it is infeasible to find r such that $(r, H(r)) \in \mathcal{R}$.

Challenge 2: instantiating RO

- **Needed property:** “cannot do better than brute-force”.
- Classical formalization:

Correlation Intractability (CI)

For a sufficiently sparse relation $\mathcal{R}$, it is infeasible to find r such that $(r, H(r)) \in \mathcal{R}$.

- **[Zha16]:** $H = \text{UHF} \circ \text{ELF}$ satisfies this notion for $\mathcal{R}$ depending only on the output $H(r)$.

Challenge 2: instantiating RO

- **Needed property:** “cannot do better than brute-force”.
- Classical formalization:

Correlation Intractability (CI)

For a sufficiently sparse relation $\mathcal{R}$, it is infeasible to find r such that $(r, H(r)) \in \mathcal{R}$.

- **[Zha16]:** $H = \text{UHF} \circ \text{ELF}$ satisfies this notion for $\mathcal{R}$ depending only on the output $H(r)$.
- One could try to encode anamorphic resistance as output intractability:

$$\mathcal{R}(y) = [\text{AE.Dec}(\text{sk}, \text{dk}, \text{Enc}(\text{pk}, m; y) = \hat{m})].$$

Challenge 2: instantiating RO

- **Needed property:** “cannot do better than brute-force”.
- Classical formalization:

Correlation Intractability (CI)

For a sufficiently sparse relation $\mathcal{R}$, it is infeasible to find r such that $(r, H(r)) \in \mathcal{R}$.

- **[Zha16]:** $H = \text{UHF} \circ \text{ELF}$ satisfies this notion for $\mathcal{R}$ depending only on the output $H(r)$.
- One could try to encode anamorphic resistance as output intractability:

$$\mathcal{R}(y) = [\text{AE.Dec}(\text{sk}, \text{dk}, \text{Enc}(\text{pk}, m; y) = \hat{m})].$$

- But now sparsity needs to be argued against all anamorphic schemes at once.

Challenge 2: instantiating RO

- **Needed property:** “cannot do better than brute-force”.
- Classical formalization:

Correlation Intractability (CI)

For a sufficiently sparse relation $\mathcal{R}$, it is infeasible to find r such that $(r, H(r)) \in \mathcal{R}$.

- **[Zha16]:** $H = \text{UHF} \circ \text{ELF}$ satisfies this notion for $\mathcal{R}$ depending only on the output $H(r)$.
- One could try to encode anamorphic resistance as output intractability:

$$\mathcal{R}(y) = [\text{AE.Dec}(\text{sk}, \text{dk}, \text{Enc}(\text{pk}, m; y) = \hat{m})].$$

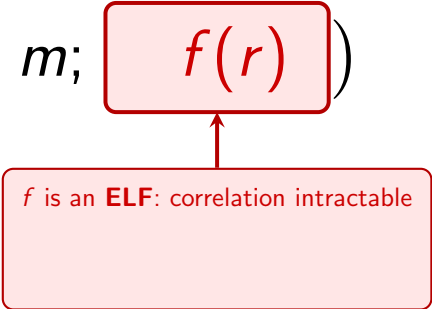
- But now sparsity needs to be argued against all anamorphic schemes at once.
- Easier to work directly with the ELF and UHF for a *fixed* anamorphic scheme.

$$ct = \text{Enc}(pk, m; \boxed{r})$$

The diagram illustrates the input r to the encryption function Enc . A light green rounded rectangle labeled r is shown as an input to the function. Below it, another light green rounded rectangle labeled "fresh random coins" has a green arrow pointing upwards to the r box, indicating that r is derived from fresh random coins.

$$\text{ct} = \text{Enc}(\text{pk}, m; \boxed{f(r)})$$

f is an **ELF**: correlation intractable



$$\text{ct} = \text{Enc}(\text{pk}, m; \boxed{f(r)})$$

f is an **ELF**: correlation intractable
 $\Rightarrow$ hard to find a useful r

$$\text{ct} = \text{Enc}(\text{pk}, m; \boxed{f(r)})$$

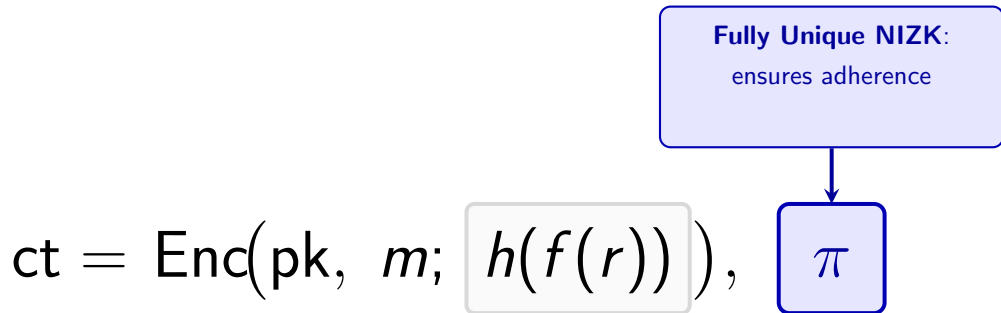
f is an **ELF**: correlation intractable
 $\Rightarrow$ hard to find a useful r
instantiated in **injective mode**

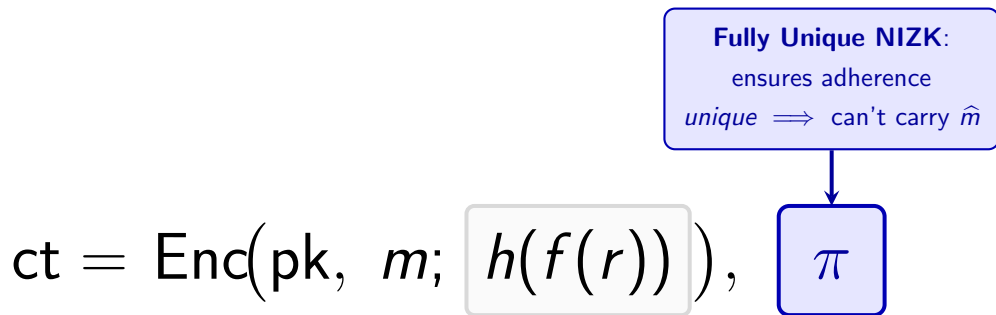
$$\text{ct} = \text{Enc}(\text{pk}, m; \boxed{h(f(r))})$$

h is a **UHF** (randomness extractor)

$$\text{ct} = \text{Enc}(\text{pk}, m; \boxed{h(f(r))})$$

h is a **UHF** (randomness extractor)
 $h(f(r)) \approx_c \text{uniform} \Rightarrow \text{CPA security}$





- 1 Introduction and definitions
- 2 Construction 1: conceptual
- 3 Proof of Anamorphic Resistance**
- 4 Construction 2: practical
- 5 Construction 3: Definitive ARE
- 6 Summary

What we need to show

What we need to show

- 1 IND-CPA security.

What we need to show

① IND-CPA security.

- Standard: cannot distinguish encryptions of m_0 vs. m_1 .

What we need to show

1 IND-CPA security.

- Standard: cannot distinguish encryptions of m_0 vs. m_1 .

2 Anamorphic resistance.

What we need to show

1 IND-CPA security.

- Standard: cannot distinguish encryptions of m_0 vs. m_1 .

2 Anamorphic resistance.

- Any anamorphic triplet (AE.Gen, AE.Enc, AE.Dec) for this scheme has a **polynomial** anamorphic message space $\hat{M}$.

What we need to show

1 IND-CPA security.

- Standard: cannot distinguish encryptions of m_0 vs. m_1 .

2 Anamorphic resistance.

- Any anamorphic triplet (AE.Gen, AE.Enc, AE.Dec) for this scheme has a **polynomial** anamorphic message space $\hat{M}$.
- Otherwise:

What we need to show

1 IND-CPA security.

- Standard: cannot distinguish encryptions of m_0 vs. m_1 .

2 Anamorphic resistance.

- Any anamorphic triplet (AE.Gen, AE.Enc, AE.Dec) for this scheme has a **polynomial** anamorphic message space $\hat{M}$.
- Otherwise:
 - Lossy mode: AE must be broken.

What we need to show

1 IND-CPA security.

- Standard: cannot distinguish encryptions of m_0 vs. m_1 .

2 Anamorphic resistance.

- Any anamorphic triplet (AE.Gen, AE.Enc, AE.Dec) for this scheme has a **polynomial** anamorphic message space $\hat{M}$.
- Otherwise:
 - Lossy mode: AE must be broken.
 - So it must also break in the injective mode (or we would distinguish the modes).

What we need to show

1 IND-CPA security.

- Standard: cannot distinguish encryptions of m_0 vs. m_1 .

2 Anamorphic resistance.

- Any anamorphic triplet (AE.Gen, AE.Enc, AE.Dec) for this scheme has a **polynomial** anamorphic message space $\hat{M}$.
- Otherwise:
 - Lossy mode: AE must be broken.
 - So it must also break in the injective mode (or we would distinguish the modes).
- Also rules out adaptive AE.

H_0
real game

Challenge ciphertext

$$ct^* = (\text{Enc}(pk, m_b; h(f(r))), \pi_{\text{real}})$$

IND-CPA security

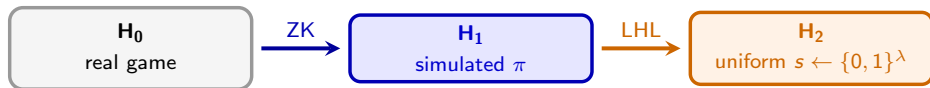


Challenge ciphertext

$$ct^* = (\text{Enc}(pk, m_b; h(f(r))), \pi_{\text{sim}})$$

- **ZK:** simulate π (needed before touching $h(f(r))$)

IND-CPA security

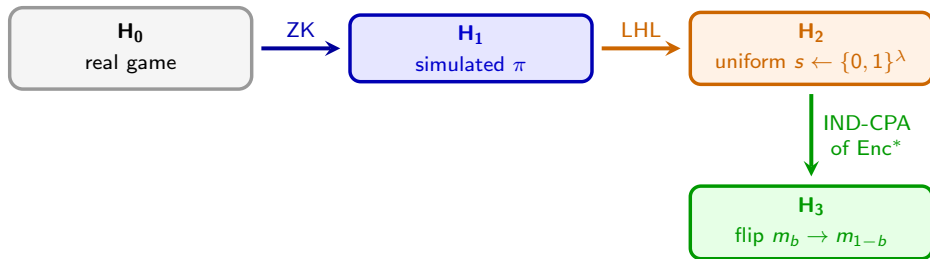


$$\text{ct}^* = (\text{Enc}(\text{pk}, \boxed{m_b}; \boxed{s}), \boxed{\pi_{\text{sim}}})$$

Challenge ciphertext

- **ZK**: simulate π (needed before touching $h(f(r))$)
- **LHL**: $h(f(r)) \approx_s s$ (since $r \leftarrow \{0, 1\}^{3\lambda}$ and f injective)

IND-CPA security



Challenge ciphertext

$$\text{ct}^* = (\text{Enc}(\text{pk}, \boxed{m_{1-b}}; \boxed{s}), \boxed{\pi_{\text{sim}}})$$

- **ZK:** simulate π (needed before touching $h(f(r))$)
- **LHL:** $h(f(r)) \approx_s s$ (since $r \leftarrow \{0, 1\}^{3\lambda}$ and f injective)
- **IND-CPA:** now s is uniform, so switch $m_b \leftrightarrow m_{1-b}$

Anamorphic Resistance: intuition

$$\text{ct} = \text{Enc}(\text{pk}, m; h(f(r))) + \pi$$

- **ELF intuition:**

Anamorphic Resistance: intuition

$$\text{ct} = \text{Enc}(\text{pk}, m; h(f(r))) + \pi$$

- **ELF intuition:** $h \circ f$ behaves like a Random Oracle.

Anamorphic Resistance: intuition

$$\text{ct} = \text{Enc}(\text{pk}, m; h(f(r))) + \pi$$

- **ELF intuition:** $h \circ f$ behaves like a Random Oracle.
- **FUNIZK π :**

Anamorphic Resistance: intuition

$$\text{ct} = \text{Enc}(\text{pk}, m; h(f(r))) + \pi$$

- **ELF intuition:** $h \circ f$ behaves like a Random Oracle.
- **FUNIZK π :** adherence is enforced; π carries no extra channel.

Anamorphic Resistance: intuition

$$\text{ct} = \text{Enc}(\text{pk}, m; h(f(r))) + \pi$$

- **ELF intuition:** $h \circ f$ behaves like a Random Oracle.
- **FUNIZK π :** adherence is enforced; π carries no extra channel.
- **So what can a citizen do?**

Anamorphic Resistance: intuition

$$\text{ct} = \text{Enc}(\text{pk}, m; h(f(r))) + \pi$$

- **ELF intuition:** $h \circ f$ behaves like a Random Oracle.
- **FUNIZK π :** adherence is enforced; π carries no extra channel.
- **So what can a citizen do?** only rejection-sample over r .

Anamorphic Resistance: intuition

$$\text{ct} = \text{Enc}(\text{pk}, m; h(f(r))) + \pi$$

- **ELF intuition:** $h \circ f$ behaves like a Random Oracle.
- **FUNIZK π :** adherence is enforced; π carries no extra channel.
- **So what can a citizen do?** only rejection-sample over r .
- **Formal proof:**

Anamorphic Resistance: intuition

$$\text{ct} = \text{Enc}(\text{pk}, m; h(f(r))) + \pi$$

- **ELF intuition:** $h \circ f$ behaves like a Random Oracle.
- **FUNIZK π :** adherence is enforced; π carries no extra channel.
- **So what can a citizen do?** only rejection-sample over r .
- **Formal proof:** build a distinguisher between the ELF modes.

Distinguisher intuition

Real world: f injective

r

$h(f(r))$

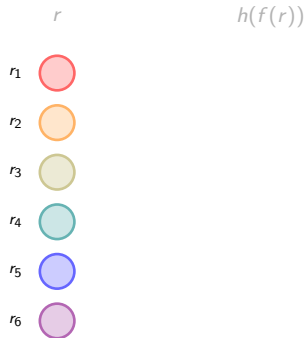
Proof: f lossy, image size R

r

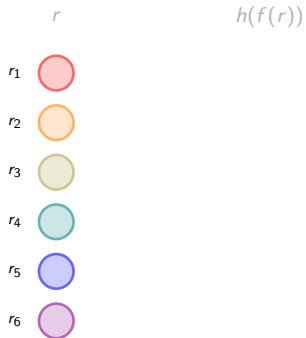
$h(f(r))$

Distinguisher intuition

Real world: f injective

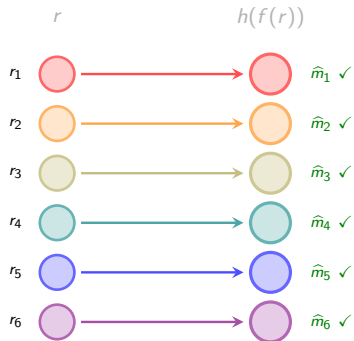


Proof: f lossy, image size R

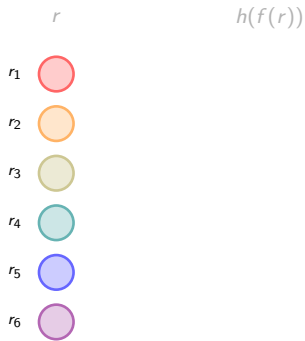


Distinguisher intuition

Real world: f injective

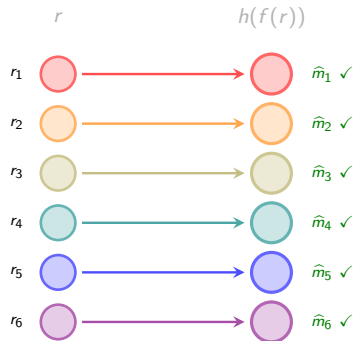


Proof: f lossy, image size R



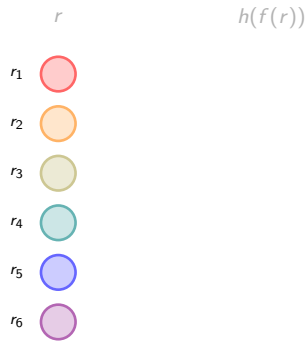
Distinguisher intuition

Real world: f injective



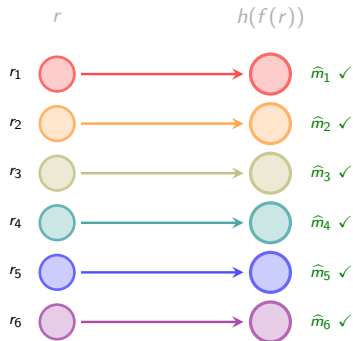
each $\hat{m}$ usually gets a unique slot
 $\Rightarrow$ anamorphic encoding works

Proof: f lossy, image size R



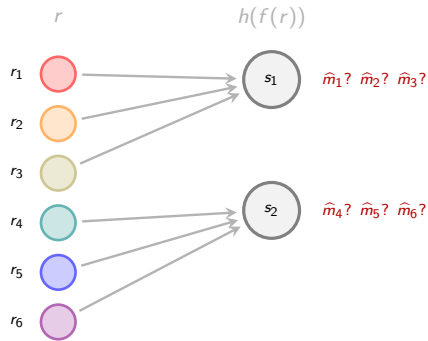
Distinguisher intuition

Real world: f injective



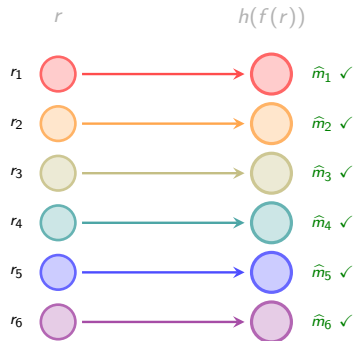
each $\hat{m}$ usually gets a unique slot
 $\Rightarrow$ anamorphic encoding works

Proof: f lossy, image size R



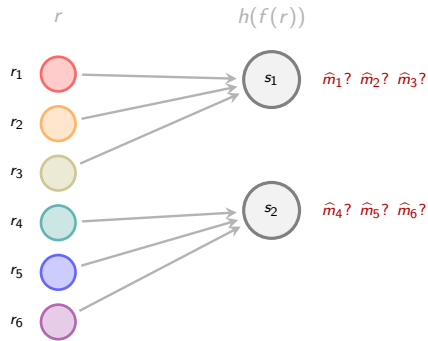
Distinguisher intuition

Real world: f injective



each $\hat{m}$ usually gets a unique slot
 $\Rightarrow$ anamorphic encoding works

Proof: f lossy, image size R



only $R = \text{poly}(\lambda)$ slots
 $\Rightarrow |\hat{M}|/R$ collisions per slot
 $\Rightarrow$ cannot decrypt!

The ELF Distinguisher $\mathcal{D}(f)$

Inject f into $\text{pp} = (f, h, \text{crs}, \text{pp}^*)$
 $(\text{pk}, \text{sk}, \text{dk}) \leftarrow \text{AE.Gen}(\text{pp})$
 $m \leftarrow \$ M, \hat{m} \leftarrow \$ \hat{M}$

The ELF Distinguisher $\mathcal{D}(f)$

Inject f into $\text{pp} = (f, h, \text{crs}, \text{pp}^*)$
 $(\text{pk}, \text{sk}, \text{dk}) \leftarrow \text{AE.Gen}(\text{pp})$
 $m \leftarrow \$ M, \hat{m} \leftarrow \$ \hat{M}$

$\text{ct} = (e, \pi) \leftarrow \text{AE.Enc}(\text{pk}, \text{dk}, m, \hat{m}).$
 $\tilde{m} := \text{AE.Dec}(\text{sk}, \text{dk}, \text{ct}).$

The ELF Distinguisher $\mathcal{D}(f)$

Inject f into $\text{pp} = (f, h, \text{crs}, \text{pp}^*)$
 $(\text{pk}, \text{sk}, \text{dk}) \leftarrow \text{AE.Gen}(\text{pp})$
 $m \leftarrow \$ M, \hat{m} \leftarrow \$ \hat{M}$

$\text{ct} = (e, \pi) \leftarrow \text{AE.Enc}(\text{pk}, \text{dk}, m, \hat{m}).$
 $\tilde{m} := \text{AE.Dec}(\text{sk}, \text{dk}, \text{ct}).$

(NV) π verifies $\wedge$ **(DC)** e decrypts correctly $\wedge$ **(KC)** keys are valid $\wedge$ **(AC)** $\tilde{m} = \hat{m}.$

The ELF Distinguisher $\mathcal{D}(f)$

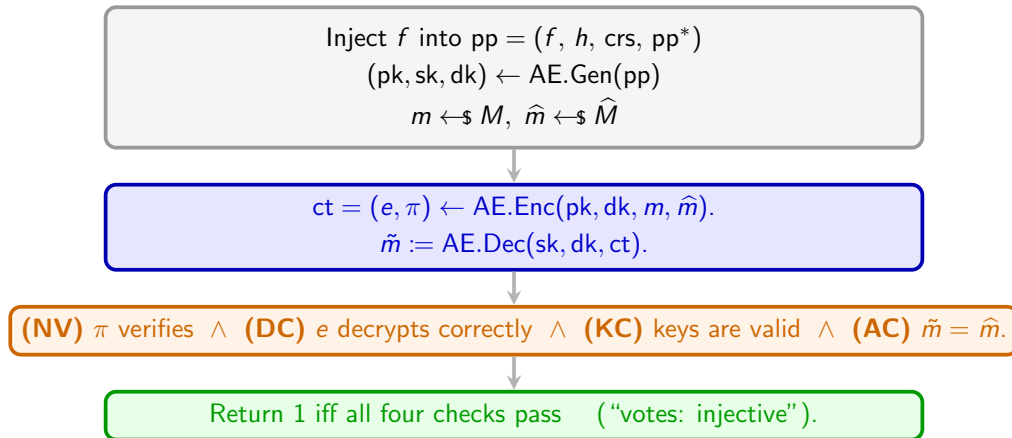
Inject f into $pp = (f, h, crs, pp^*)$
 $(pk, sk, dk) \leftarrow \text{AE.Gen}(pp)$
 $m \leftarrow \$ M, \hat{m} \leftarrow \$ \hat{M}$

$ct = (e, \pi) \leftarrow \text{AE.Enc}(pk, dk, m, \hat{m}).$
 $\tilde{m} := \text{AE.Dec}(sk, dk, ct).$

(NV) π verifies $\wedge$ **(DC)** e decrypts correctly $\wedge$ **(KC)** keys are valid $\wedge$ **(AC)** $\tilde{m} = \hat{m}.$

Return 1 iff all four checks pass ("votes: injective").

The ELF Distinguisher $\mathcal{D}(f)$



- **(AC)** is what separates the modes: succeeds injective, fails lossy.

The ELF Distinguisher $\mathcal{D}(f)$

Inject f into $pp = (f, h, crs, pp^*)$
 $(pk, sk, dk) \leftarrow \text{AE.Gen}(pp)$
 $m \leftarrow \$ M, \hat{m} \leftarrow \$ \hat{M}$

$ct = (e, \pi) \leftarrow \text{AE.Enc}(pk, dk, m, \hat{m}).$
 $\tilde{m} := \text{AE.Dec}(sk, dk, ct).$

(NV) π verifies $\wedge$ **(DC)** e decrypts correctly $\wedge$ **(KC)** keys are valid $\wedge$ **(AC)** $\tilde{m} = \hat{m}.$

Return 1 iff all four checks pass (“votes: injective”).

- **(AC)** is what separates the modes: succeeds injective, fails lossy.
- **Why (NV), (DC), (KC)?** In case AE distinguishes itself.

Analysis: injective vs. lossy mode

Injective mode ($f = f_{\text{inj}}$):

Analysis: injective vs. lossy mode

Injective mode ($f = f_{\text{inj}}$):

- Well-formedness checks hold by *semi-adaptive* security (dictator can verify each predicate).

Analysis: injective vs. lossy mode

Injective mode ($f = f_{\text{inj}}$):

- Well-formedness checks hold by *semi-adaptive* security (dictator can verify each predicate).
- Anamorphic correctness by assumption.

Analysis: injective vs. lossy mode

Injective mode ($f = f_{\text{inj}}$):

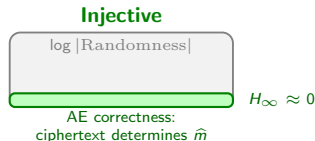
- Well-formedness checks hold by *semi-adaptive* security (dictator can verify each predicate).
- Anamorphic correctness by assumption.
- $\Rightarrow \Pr[\mathcal{D}(f_{\text{inj}}) = 1] \approx 1.$

Analysis: injective vs. lossy mode

Injective mode ($f = f_{\text{inj}}$):

- Well-formedness checks hold by *semi-adaptive* security (dictator can verify each predicate).
- Anamorphic correctness by assumption.
- $\Rightarrow \Pr[\mathcal{D}(f_{\text{inj}}) = 1] \approx 1$.

Uncertainty about $\hat{m}$:



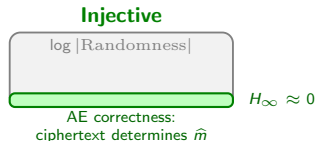
Analysis: injective vs. lossy mode

Injective mode ($f = f_{\text{inj}}$):

- Well-formedness checks hold by *semi-adaptive* security (dictator can verify each predicate).
- Anamorphic correctness by assumption.
- $\Rightarrow \Pr[\mathcal{D}(f_{\text{inj}}) = 1] \approx 1$.

Lossy mode ($f = f_{\text{lossy}}$, image size R):

Uncertainty about $\hat{m}$:



Analysis: injective vs. lossy mode

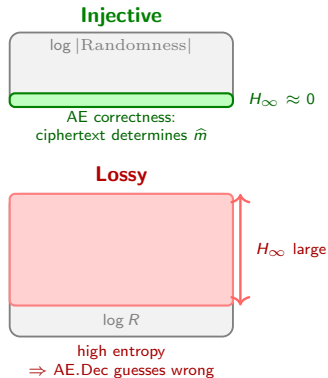
Injective mode ($f = f_{\text{inj}}$):

- Well-formedness checks hold by *semi-adaptive* security (dictator can verify each predicate).
- Anamorphic correctness by assumption.
- $\Rightarrow \Pr[\mathcal{D}(f_{\text{inj}}) = 1] \approx 1$.

Lossy mode ($f = f_{\text{lossy}}$, image size R):

- Many collisions $\Rightarrow$ ciphertext underdetermines $\hat{m}$:

Uncertainty about $\hat{m}$:



Analysis: injective vs. lossy mode

Injective mode ($f = f_{\text{inj}}$):

- Well-formedness checks hold by *semi-adaptive* security (dictator can verify each predicate).
- Anamorphic correctness by assumption.
- $\Rightarrow \Pr[\mathcal{D}(f_{\text{inj}}) = 1] \approx 1$.

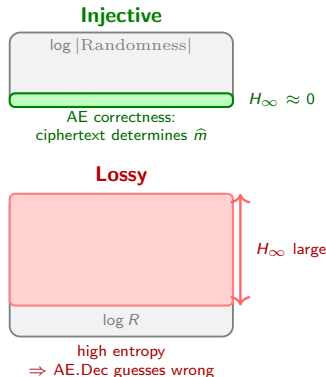
Lossy mode ($f = f_{\text{lossy}}$, image size R):

- Many collisions $\Rightarrow$ ciphertext underdetermines $\hat{m}$:

Core lemma

$$H_{\infty}(\hat{m} \mid \text{sk}, \text{dk}, \text{ct}) \geq \log |\hat{M}| - \log R + \log \Pr[\text{ct valid}].$$

Uncertainty about $\hat{m}$:



Analysis: injective vs. lossy mode

Injective mode ($f = f_{\text{inj}}$):

- Well-formedness checks hold by *semi-adaptive* security (dictator can verify each predicate).
- Anamorphic correctness by assumption.
- $\Rightarrow \Pr[\mathcal{D}(f_{\text{inj}}) = 1] \approx 1$.

Lossy mode ($f = f_{\text{lossy}}$, image size R):

- Many collisions $\Rightarrow$ ciphertext underdetermines $\hat{m}$:

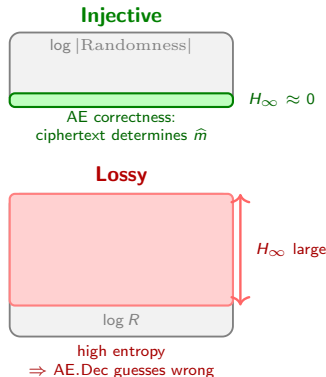
Core lemma

$$H_{\infty}(\hat{m} \mid \text{sk}, \text{dk}, \text{ct}) \geq \log |\hat{M}| - \log R + \log \Pr[\text{ct valid}].$$

Consequence: low decoding probability

$$\Pr[\mathcal{D}(f_{\text{lossy}}) = 1] \leq R/|\hat{M}| + \text{negl}(\lambda).$$

Uncertainty about $\hat{m}$:



Anamorphic Resistance — conclusion

- Combining both modes:

$$\text{Adv}(\mathcal{D}) \geq \Pr[\mathcal{D}(f_{\text{inj}}) = 1] - \Pr[\mathcal{D}(f_{\text{lossy}}) = 1] \geq 1 - \frac{R}{|\widehat{M}|} - \text{negl}(\lambda).$$

Anamorphic Resistance — conclusion

- Combining both modes:

$$\text{Adv}(\mathcal{D}) \geq \Pr[\mathcal{D}(f_{\text{inj}}) = 1] - \Pr[\mathcal{D}(f_{\text{lossy}}) = 1] \geq 1 - \frac{R}{|\widehat{M}|} - \text{negl}(\lambda).$$

- ELF fine-grained security:** take $R = \text{poly}(\lambda)$, such that $\mathcal{D}$ has advantage $\leq 1/2$ in distinguishing f_{inj} from f_{lossy} .¹

¹ R depends on the runtime of $\mathcal{D}$.

Anamorphic Resistance — conclusion

- Combining both modes:

$$\text{Adv}(\mathcal{D}) \geq \Pr[\mathcal{D}(f_{\text{inj}}) = 1] - \Pr[\mathcal{D}(f_{\text{lossy}}) = 1] \geq 1 - \frac{R}{|\widehat{M}|} - \text{negl}(\lambda).$$

- ELF fine-grained security:** take $R = \text{poly}(\lambda)$, such that $\mathcal{D}$ has advantage $\leq 1/2$ in distinguishing f_{inj} from f_{lossy} .¹
- Plug in:

$$\frac{1}{2} \geq 1 - \frac{R}{|\widehat{M}|} - \text{negl}(\lambda),$$

¹ R depends on the runtime of $\mathcal{D}$.

Anamorphic Resistance — conclusion

- Combining both modes:

$$\text{Adv}(\mathcal{D}) \geq \Pr[\mathcal{D}(f_{\text{inj}}) = 1] - \Pr[\mathcal{D}(f_{\text{lossy}}) = 1] \geq 1 - \frac{R}{|\widehat{M}|} - \text{negl}(\lambda).$$

- ELF fine-grained security:** take $R = \text{poly}(\lambda)$, such that $\mathcal{D}$ has advantage $\leq 1/2$ in distinguishing f_{inj} from f_{lossy} .¹
- Plug in:

$$\frac{1}{2} \geq 1 - \frac{R}{|\widehat{M}|} - \text{negl}(\lambda),$$

hence

$$|\widehat{M}| \leq 2R + \text{negl}(\lambda) = \text{poly}(\lambda). \quad \square$$

¹ R depends on the runtime of $\mathcal{D}$.

- 1 Introduction and definitions
- 2 Construction 1: conceptual
- 3 Proof of Anamorphic Resistance
- 4 Construction 2: practical**
- 5 Construction 3: Definitive ARE
- 6 Summary

- The construction above is conceptually clear, but relies on $i\mathcal{O}$.

- The construction above is conceptually clear, but relies on $i\mathcal{O}$.
- Fully Unique NIZKs in enough generality imply witness encryption.

- The construction above is conceptually clear, but relies on $i\mathcal{O}$.
- Fully Unique NIZKs in enough generality imply witness encryption.
- **Observation:** the challenge is to prove compliance.

- The construction above is conceptually clear, but relies on $i\mathcal{O}$.
- Fully Unique NIZKs in enough generality imply witness encryption.
- **Observation:** the challenge is to prove compliance.
- **Idea:** check it with a dictator-owned trapdoor.

- The construction above is conceptually clear, but relies on $i\mathcal{O}$.
- Fully Unique NIZKs in enough generality imply witness encryption.
- **Observation:** the challenge is to prove compliance.
- **Idea:** check it with a dictator-owned trapdoor.
- **Tool:** trapdoors for ELF. [Zha19]

$$\text{ct} = \text{Enc}(\text{pk}^*, m; \boxed{h(f(r))}), \quad \boxed{\phantom{\text{ct}}}$$

↑

same as before: **ELF** + **UHF**

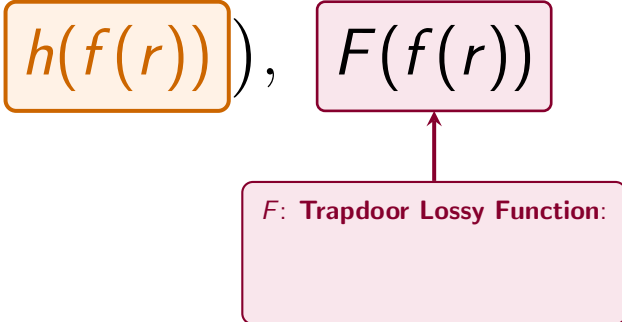
$$\text{ct} = \text{Enc}(\text{pk}^*, m; \boxed{h(f(r))}), \quad \boxed{\phantom{\text{ct}}}$$

↑

same as before: **ELF** + **UHF**
but dictator has td_f

$$\text{ct} = \text{Enc}(\text{pk}^*, m; \boxed{h(f(r))}), \quad \boxed{F(f(r))}$$

F : Trapdoor Lossy Function:



$$\text{ct} = \text{Enc}(\text{pk}^*, m; \boxed{h(f(r))}), \quad \boxed{F(f(r))}$$

F : Trapdoor Lossy Function:
 $\approx$ deterministic encryption

$$\text{ct} = \text{Enc}(\text{pk}^*, m; \boxed{h(f(r))}), \quad \boxed{F(f(r))}$$

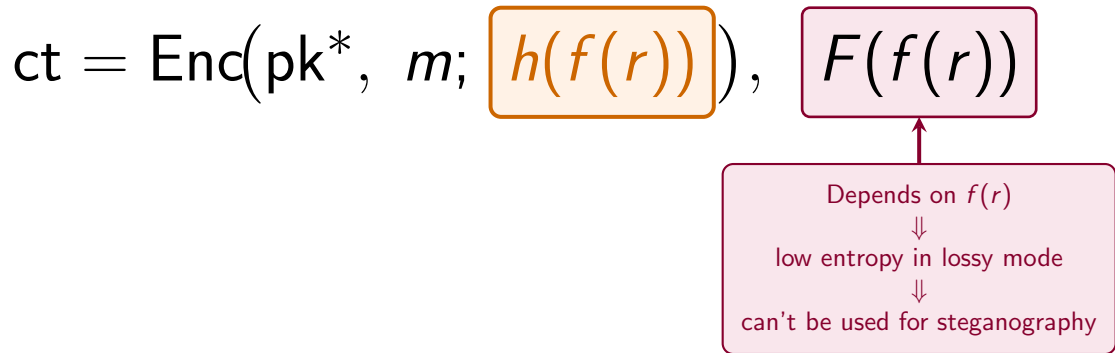
F : Trapdoor Lossy Function:
 $\approx$ deterministic encryption
“sends” $f(r)$ to the dictator

$$\text{ct} = \text{Enc}(\text{pk}^*, m; \boxed{h(f(r))}), \quad \boxed{F(f(r))}$$

Depends on $f(r)$

$$\text{ct} = \text{Enc}(\text{pk}^*, m; \boxed{h(f(r))}), \quad \boxed{F(f(r))}$$

Depends on $f(r)$
↓
low entropy in lossy mode



Catching non-compliant citizens

ciphertext $c = (e, v)$

Dictator



Catching non-compliant citizens

ciphertext $c = (e, v)$ supposed to be $e = \text{Enc}(\text{pk}^*, m; h(\rho))$, $v = F(f(r))$

Dictator



Catching non-compliant citizens

ciphertext $c = (e, v)$ supposed to be $e = \text{Enc}(\text{pk}^*, m; h(\rho))$, $v = F(f(r))$



Step 1: recover $\rho = F^{-1}(v)$

Dictator



Catching non-compliant citizens

ciphertext $c = (e, v)$ supposed to be $e = \text{Enc}(\text{pk}^*, m; h(\rho))$, $v = F(f(r))$

Step 1: recover $\rho = F^{-1}(v)$ (via TLF trapdoor for F)

Dictator



Catching non-compliant citizens

ciphertext $c = (e, v)$ supposed to be $e = \text{Enc}(\text{pk}^*, m; h(\rho))$, $v = F(f(r))$

Step 1: recover $\rho = F^{-1}(v)$ (via TLF trapdoor for F)

Step 2: recover $r = f^{-1}(\rho)$

Dictator



Catching non-compliant citizens

ciphertext $c = (e, v)$ supposed to be $e = \text{Enc}(\text{pk}^*, m; h(\rho))$, $v = F(f(r))$

Step 1: recover $\rho = F^{-1}(v)$ (via TLF trapdoor for F)

Step 2: recover $r = f^{-1}(\rho)$ (via TELF trapdoor for f)

Dictator



Catching non-compliant citizens

ciphertext $c = (e, v)$ supposed to be $e = \text{Enc}(\text{pk}^*, m; h(\rho))$, $v = F(f(r))$

Step 1: recover $\rho = F^{-1}(v)$ (via TLF trapdoor for F)

Step 2: recover $r = f^{-1}(\rho)$ (via TELF trapdoor for f)

Step 3: decrypt $m = \text{Dec}(\text{sk}, e)$

Dictator



Catching non-compliant citizens

ciphertext $c = (e, v)$ supposed to be $e = \text{Enc}(\text{pk}^*, m; h(\rho))$, $v = F(f(r))$

Step 1: recover $\rho = F^{-1}(v)$ (via TLF trapdoor for F)

Step 2: recover $r = f^{-1}(\rho)$ (via TELF trapdoor for f)

Step 3: decrypt $m = \text{Dec}(\text{sk}, e)$ (dictator knows sk)

Dictator



Catching non-compliant citizens

ciphertext $c = (e, v)$ supposed to be $e = \text{Enc}(\text{pk}^*, m; h(\rho))$, $v = F(f(r))$

Step 1: recover $\rho = F^{-1}(v)$ (via TLF trapdoor for F)

Step 2: recover $r = f^{-1}(\rho)$ (via TELF trapdoor for f)

Step 3: decrypt $m = \text{Dec}(\text{sk}, e)$ (dictator knows sk)

Step 4: check $e = \text{Enc}(\text{pk}^*, m; h(\rho))$ and $v = F(f(r))$

Dictator



Catching non-compliant citizens

ciphertext $c = (e, v)$ supposed to be $e = \text{Enc}(\text{pk}^*, m; h(\rho))$, $v = F(f(r))$

Step 1: recover $\rho = F^{-1}(v)$ (via TLF trapdoor for F)

Step 2: recover $r = f^{-1}(\rho)$ (via TELF trapdoor for f)

Step 3: decrypt $m = \text{Dec}(\text{sk}, e)$ (dictator knows sk)

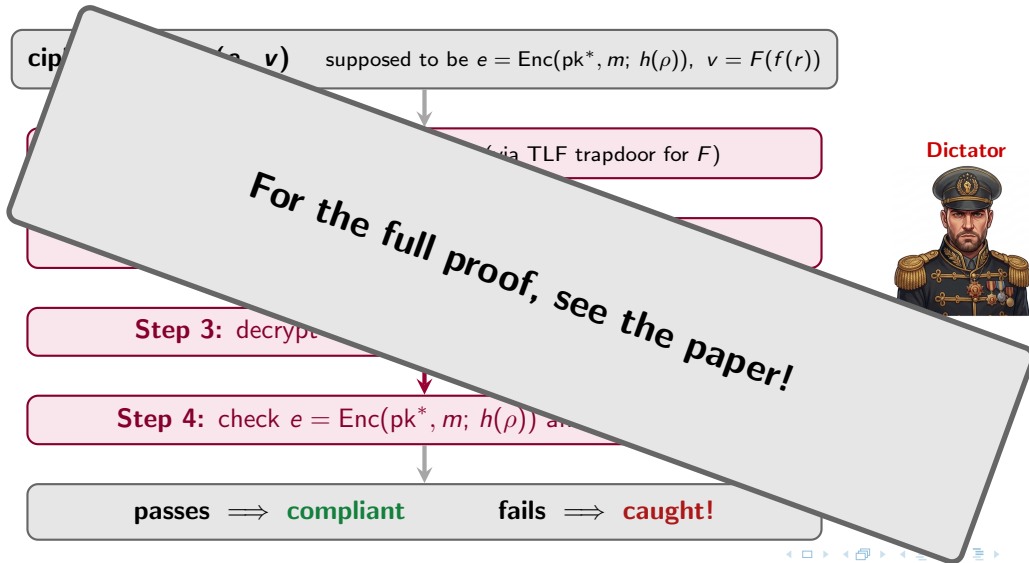
Step 4: check $e = \text{Enc}(\text{pk}^*, m; h(\rho))$ and $v = F(f(r))$

passes $\implies$ **compliant** fails $\implies$ **caught!**

Dictator



Catching non-compliant citizens



- 1 Introduction and definitions
- 2 Construction 1: conceptual
- 3 Proof of Anamorphic Resistance
- 4 Construction 2: practical
- 5 Construction 3: Definitive ARE**
- 6 Summary

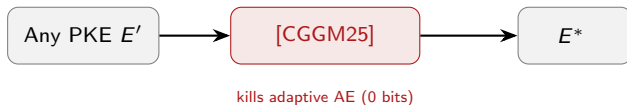
Definitive ARE: the worst of both worlds

Aim: resistance to adaptive and semi-adaptive AE

Any PKE E'

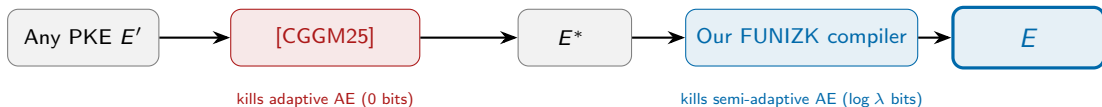
Definitive ARE: the worst of both worlds

Aim: resistance to adaptive and semi-adaptive AE



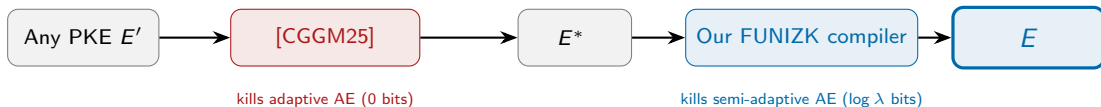
Definitive ARE: the worst of both worlds

Aim: resistance to adaptive and semi-adaptive AE



Definitive ARE: the worst of both worlds

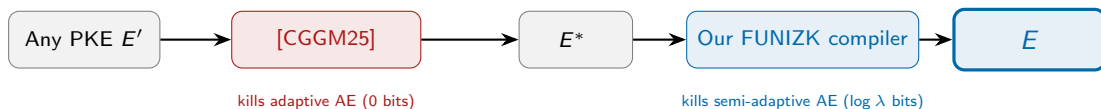
Aim: resistance to adaptive and semi-adaptive AE



- E^* does not admit adaptive AE $\implies$ same holds for E

Definitive ARE: the worst of both worlds

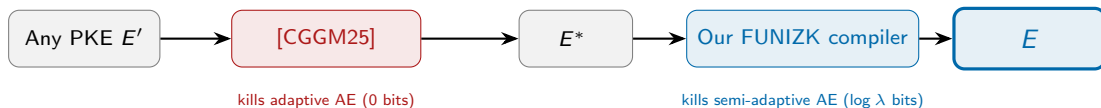
Aim: resistance to adaptive and semi-adaptive AE



- E^* does not admit adaptive AE $\implies$ same holds for E
(otherwise, we would simulate the NIZK)

Definitive ARE: the worst of both worlds

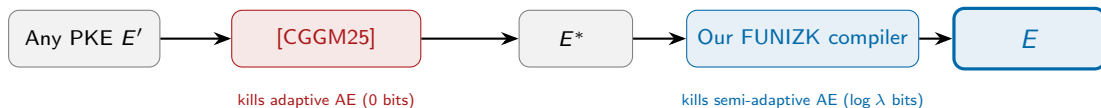
Aim: resistance to adaptive and semi-adaptive AE



- E^* does not admit adaptive AE $\implies$ same holds for E (otherwise, we would simulate the NIZK)
- The construction doesn't go through with the TELF scheme

Definitive ARE: the worst of both worlds

Aim: resistance to adaptive and semi-adaptive AE



- E^* does not admit adaptive AE $\implies$ same holds for E (otherwise, we would simulate the NIZK)
- The construction doesn't go through with the TELF scheme ($F(f(r))$ could be rejection sampled)

- 1 Introduction and definitions
- 2 Construction 1: conceptual
- 3 Proof of Anamorphic Resistance
- 4 Construction 2: practical
- 5 Construction 3: Definitive ARE
- 6 Summary

Summary

Dictator wins!



Alice

× NO PRIVACY ×



Bob

Summary

Dictator wins!



× NO PRIVACY ×



Bob

ELF+FUNIZK

- ✓ ROM-free (needs *iO*)
- ✓ Enables Definitive ARE
- ✓ Conceptually simple

Dictator wins!



× NO PRIVACY ×



Bob

ELF+FUNIZK

- ✓ ROM-free (needs $i\mathcal{O}$)
- ✓ Enables Definitive ARE
- ✓ Conceptually simple

Trapdoor ELF

- ✓ ROM-free, $i\mathcal{O}$ -free
- ✓ Exp-DDH only
- ✓ Practical

Dictator wins!



× **NO PRIVACY** ×



ELF+FUNIZK

- ✓ ROM-free (needs $i\mathcal{O}$)
- ✓ Enables Definitive ARE
- ✓ Conceptually simple

Trapdoor ELF

- ✓ ROM-free, $i\mathcal{O}$ -free
- ✓ Exp-DDH only
- ✓ Practical

Definitive ARE

- ✓ Semi-adap: $\leq O(\log \lambda)$ bits
- ✓ Adaptive: 0 bits
- ✓ Theoretically optimal

Dictator wins!



× NO PRIVACY ×



ELF+FUNIZK

- ✓ ROM-free (needs $i\mathcal{O}$)
- ✓ Enables Definitive ARE
- ✓ Conceptually simple

Trapdoor ELF

- ✓ ROM-free, $i\mathcal{O}$ -free
- ✓ Exp-DDH only
- ✓ Practical

Definitive ARE

- ✓ Semi-adap: $\leq O(\log \lambda)$ bits
- ✓ Adaptive: 0 bits
- ✓ Theoretically optimal

Dictator wins!



× NO PRIVACY ×



ELF+FUNIZK

- ✓ ROM-free (needs $i\mathcal{O}$)
- ✓ Enables Definitive ARE
- ✓ Conceptually simple

Trapdoor ELF

- ✓ ROM-free, $i\mathcal{O}$ -free
- ✓ Exp-DDH only
- ✓ Practical

Definitive ARE

- ✓ Semi-adap: $\leq O(\log \lambda)$ bits
- ✓ Adaptive: 0 bits
- ✓ Theoretically optimal

Our contribution: a very strong impossibility in this dictator model.

Dictator wins!



× NO PRIVACY ×

ELF+FUNIZK

- ✓ ROM-free (needs $i\mathcal{O}$)
- ✓ Enables Definitive ARE
- ✓ Conceptually simple

Trapdoor ELF

- ✓ ROM-free, $i\mathcal{O}$ -free
- ✓ Exp-DDH only
- ✓ Practical

Definitive ARE

- ✓ Semi-adap: $\leq O(\log \lambda)$ bits
- ✓ Adaptive: 0 bits
- ✓ Theoretically optimal

Our contribution: a very strong impossibility in this dictator model.

Takeaway: existence of anamorphic primitives may not be enough.

Dictator wins!



× NO PRIVACY ×

ELF+FUNIZK

- ✓ ROM-free (needs $i\mathcal{O}$)
- ✓ Enables Definitive ARE
- ✓ Conceptually simple

Trapdoor ELF

- ✓ ROM-free, $i\mathcal{O}$ -free
- ✓ Exp-DDH only
- ✓ Practical

Definitive ARE

- ✓ Semi-adap: $\leq O(\log \lambda)$ bits
- ✓ Adaptive: 0 bits
- ✓ Theoretically optimal

Our contribution: a very strong impossibility in this dictator model.

Takeaway: existence of anamorphic primitives may not be enough.

Open question: can we refine the model to help citizens win?



Full paper

<https://eprint.iacr.org/2025/305>



Contact

m.mielniczuk@uw.edu.pl

Thank you for your attention!



Full version of the talk

<https://marmistrz.github.io/elfs-talk>